DCSIM HAND RECEIPT SYSTEM

Technical Handover

Repository handover & development backlog · 5 August 2026

Repository: `C:\inventoryApp` (package name `hand-receipt`)
Handover written: 2026-08-05
Assessed at revision: branch `feat/receipt-link-pin-bypass`, tip `ff4857f` plus the two commits of 2026-08-05 (`fix(security): changing a password now revokes every live session`, `docs(security): add the static security assessment of the whole tree`)
Production: Vercel (app) + Supabase (Postgres), custom domain `https://www.dcsim.us`

Points of contact
Application maintainer — SPC Xiaolan Lin, DCSIM Service Desk IT Specialist · xiaolan.lin.mil@army.mil
Developer — CDT Joshua Yang, DCSIM Intern · bubbayajo21@gmail.com (author of this document and the security assessment)

How to read this document, and what it is *not*

Everything below was verified by reading this repository. Where a claim comes from a document rather than from code, it is attributed. Where something could **not** be verified, it says so explicitly rather than guessing — those gaps are collected in §5 Known unknowns.

Nothing was executed while writing this: no build, no test run, no server, no database connection, no access to the deployed environment. Statements about *production configuration* (which environment variables are actually set in Vercel, what roles exist in Supabase) are therefore repeated from `DEPLOY.md` and `docs/SECURITY.md`, not observed.

The four documents this one sits on top of, and which you should still read:

Document	What it is for
`CLAUDE.md`	**The single richest source of architectural rationale.** Long, and worth all of it. Every "do not do this" rule in the codebase has its reasoning written down there. This handover summarizes it; it does not replace it.
`AGENTS.md`	One line, and load-bearing: this Next.js version has breaking changes vs. what you may know. Read `node_modules/next/dist/docs/` before writing framework code.
`docs/ARCHITECTURE.md`	Request flow, data model, lifecycles, the derived-readiness design, hosting topology.
`docs/SECURITY.md`	The living inventory of every security control (13 sections) plus the **Known gaps & accepted risks** register at `docs/SECURITY.md:1292`.
`SECURITY_ASSESSMENT.md`	A static multi-agent security review completed **2026-08-05**. 5 verified findings (0 Critical, 0 High, 3 Medium, 2 Low), 16 accepted risks adjudicated, 11 undocumented risks (U1–U11), 21 candidates checked and cleared. Much of §4's backlog is sourced from it.
`DEPLOY.md`	Vercel + Supabase setup, the cron, the MDM import endpoint, and the operational traps.
`CHANGELOG.md`	User-facing change history, newest first. Updating it is mandatory for any `feat:` / `fix:` — see §3.11.

1. Executive summary

What the system is

This is a **digital hand receipt and property book** for a military IT service desk — specifically the DCSIM / Hawaii ARNG G6 desk. Its job is to answer, at any moment, *who physically has which piece of IT equipment, and can we prove they signed for it.*

The paper artifact it digitizes is the **DA Form 2062** hand receipt. A technician issues equipment to a soldier or civilian, the recipient signs on-screen, and the system produces a filled, flattened DA 2062 PDF carrying both parties' details, the recipient's signature drawn vertically in the quantity column, and a QR code that opens the receipt again later. That PDF is emailed to the recipient and archived. Equipment comes back through a **return** flow — partial or full — and a full return closes the receipt, which then becomes immutable and is purged 90 days later.

Around that core sit the surfaces a working service desk actually needs:

- an **item registry** of ~1,200+ devices (make, model, serial, home unit, UIC, category, holder, notes), kept current from a nightly MDM/Intune CSV export;
- a **service queue** — one entry per item, flagged either from the receipt builder or the item page, with a service type (Reimage / Repair / Other) and an optional deadline;
- **QR labels** on every device, which open a public read-only item page;
- an **annual audit** workflow (a technician records that they physically saw the device, signing for it) that drives an accountability status;
- an **admin analytics dashboard** — audit readiness, fleet KPIs by category, DA 2062 volume, and a unit-allocation leaderboard;
- **transactional email** for every custody event, plus nightly overdue alerts.

Who uses it

Three populations, and the distinction matters because the authorization model is built around it:

1. **Service desk technicians**, each with their **own** `ADMIN` account (logins are deliberately not shared — accountability for "who processed this return" depends on it). Admins do everything: create and retire items, process returns, work the queue, manage users, run imports, record audits, set the public PIN.
2. **Standard `USER` accounts.** May read the shared inventory, create hand receipts, and edit exactly two fields on an item — the current holder's email and their position. Everything else is admin-only, enforced server-side by schema selection, not by hiding buttons.
3. **Logged-out recipients.** A soldier who was issued a laptop can look up their own hand receipt and the item page by serial number or QR scan, with no account. This is an **explicitly accepted requirement**, not an oversight — see the boxed note in `CLAUDE.md` and §3.11. Since 2026-07-22 that surface sits behind a shared 8-digit PIN; since 2026-08-04, a signed per-receipt link token in the notification email and on the printed QR lets a recipient skip the PIN for *their* receipt only.

Current state and maturity

This is a **live production system**, not a prototype. It has 565 commits since 2026-06-30, 338 TypeScript/TSX files under `src/` of which 116 are tests, 41 Prisma migrations, and a real deployment serving a government network.

The engineering discipline is unusually high for a codebase this young, and it shows in three specific places:

- **The documentation is treated as part of the change.** `CLAUDE.md` and `docs/ARCHITECTURE.md` do not just describe the code, they record *why the alternative was rejected* — including several designs that were built, shipped, and then deliberately removed (a stored readiness enum, an `isAccountedFor` flag, per-service-type SLA defaults, a `publicToken` for receipts, a "fleet status over time" chart). A future maintainer's most likely mistake is re-implementing one of those.
- **Invariants are pinned by tests rather than by convention.** The two derived-readiness implementations (TypeScript and SQL) are held in agreement by `src/modules/items/readiness.parity.test.ts`; the two `/items` query paths by `src/modules/items/items.readiness-sort.parity.test.ts`. Tests run against a real migrated Postgres, not mocks.
- **Security is inventoried, not assumed.** `docs/SECURITY.md` is 1,615 lines of control inventory with a Known-gaps register, and a CI job (`Security docs current`) fails a PR that touches a watched security file without touching it.

The 2026-08-05 security assessment (`SECURITY_ASSESSMENT.md`) found **no authorization bypass, no injection vector, and no unintended data exposure**. All 49 Server Actions and all 6 Route Handlers gate themselves as their first awaited statement. The dominant residual risk it identified is not missing controls but **documentation drift** — places where a document asserts a protection the code does not implement. One such case (self-service password change not revoking sessions) was found and fixed the same day; another (the `rls_auto_enable` database trigger, credited in three places and defined in none) is still open and is backlog item B12.

Read the scope of that result carefully, because it is easy to over-claim. The assessment was a **static source review plus a targeted dynamic pass** — the latter executed only the code behind F2/F3/F5 on a local instance. No live database or deployed environment was inspected, and **no comprehensive penetration test or full dynamic application security test has been run**. It therefore **does not establish that the system is "secure" in any absolute sense**, and it should never be cited as if it did. What it establishes is narrower and still useful: the controls that exist are implemented correctly, the access model has no *known* holes, and the team's own register of accepted risks is accurate. It complements Semgrep, dependency scanning and human review — it does not replace them, and it is a point-in-time result against revision `ff4857f`. Three of its conclusions end in "verify this in production" and remain unverified (see §5, *Known unknowns*). This caveat lives here, in the technical record, deliberately — `LEADERSHIP_BRIEF.md` reports the outcome without the methodology hedging, so this document is the one that must carry the limits.

The natural next depth is a comprehensive dynamic test. A tool such as **Strix** — a dynamic application security testing / autonomous penetration-testing tool — can be run against a **local instance (never production)** to crawl, fuzz, and probe the running app end-to-end,

far beyond the three findings verified by hand here. This is tracked as backlog B23, with the local-only guardrails stated there.

Maturity summary, honestly stated: the domain logic, authorization model, and data integrity work are mature. The weaker areas are *operational observability* (there is no authentication event log and most privileged mutations record no actor — `docs/SECURITY.md` itself names this the biggest gap), *automated test execution in CI* (the vitest suite is not run by any CI job), and *end-to-end browser coverage* (`tests/e2e/` contains exactly one spec file, `auth.spec.ts`).

---

2. Quick-start guides

2.1 For a service desk technician

This section assumes no development knowledge. Everything is done in a web browser at `https://www.dcsim.us`.

Signing in

Go to the site and click **Staff sign in** (or go to `/login`). Sign in with the email and password an admin provisioned for you. **There is no self-registration** — if you do not have an account, an existing admin creates one for you at `/admin/users`.

If Cloudflare Turnstile is switched on, the sign-in button stays disabled for a moment while the browser check completes. That is normal; submitting before it finishes would send a form the server refuses.

Two things about sessions worth knowing so they do not surprise you:

- A session lasts **10 hours from sign-in, or 4 hours idle, whichever comes first.** After that the next click returns you to the sign-in page. This is deliberate.
- **Changing your password signs you out everywhere**, including the browser you changed it in, and every other device. The sign-in page explains why rather than looking like a bug. Do it from `/account` → Change password. If you have forgotten your password, use `/forgot-password` for an emailed single-use reset link.

Finding equipment

There are two search surfaces and they do different things.

The home page (`/`) is the one a logged-out recipient uses. Type a **serial number** to find a device, or switch to receipt mode and type a **receipt number** (`HR-000123`). This search matches serial numbers and receipt numbers *only* — not device names and not people's names. If you are signed out and have not entered the access PIN, the search box does not appear.

The items list (`/items`), once you are signed in, is the real inventory view. The search box there is much broader: it matches **device name, make, model, serial number, and the name of whoever currently holds the device on an open hand receipt**. Names can be typed in either order — "doe jane" finds Jane Doe — and punctuation at the edges is ignored, so "Doe, Jane" works. Devices assigned only by the MDM import, with no hand receipt, will *not* be found by a person's name.

Beside the search box there is a **Unit (UIC)** dropdown, which is the only other filter. Every column is sortable, and you can sort by up to three columns at once by clicking headers in order. Two columns — **Readiness** and **Audit** — are computed live rather than stored, and sorting by them follows the operational sequence, not the alphabet. Note that sorting by **Audit** orders by the *badge* (compliant / overdue / never), not by how recently the audit happened; inside a badge, order comes from whatever you picked as your second sort column.

To reach a device by its QR label, **point a standard phone camera at it** — the label encodes the item's page URL (`/i/<id>` , `src/modules/items/qr.ts:33-34`), so the phone opens that page directly, no app feature involved. The **in-app scanner is a different tool**: it lives only in the hand-receipt builder, its button reads **"Scan to add"** (`src/app/receipts/new/ReceiptBuilderForm.tsx:481`), and it resolves a scan to *add that device to the receipt you are building* (`lookupScannedItem` , `src/app/actions/scan.ts`) — it is not a general "jump to a device" navigator.

Creating a hand receipt

1. From `/items` , tick the devices you are issuing (or open one device's page at `/i/<id>` and click **Create hand receipt**). Then click **Create hand receipt** in the selection bar. A receipt can hold at most 18 lines with at most 10 items per line.
2. On the builder (`/receipts/new`), fill in **both** parties. The sender is pre-filled with the device's last known receiver where there is one. Either side can be ticked **DCSIM**, which collapses that party down to just a name — use it for your own desk. A non-DCSIM party needs unit, contact number and email; **rank is optional** (the property book holds civilians and contractors). Name is asked for as "Last, First" — the field accepts anything, but filing them consistently is what makes a name search work later. The contact box offers autofill from the shared address book as you type.
3. If the **recipient is DCSIM**, a **Service** column appears with a "Needs service?" checkbox per serial — that is how kit coming *in* to the desk gets into the queue. It is deliberately hidden (and refused server-side) when the recipient is not DCSIM, because the queue is for our own work, not for equipment going out to a customer.
4. The **recipient signs on screen**. There is no separate sender signature — custody moves on the recipient's signature.
5. Submitting creates the receipt with a fresh `HR-…` number, generates the DA 2062 PDF, and sends **one** email to the customer with the PDF attached, copying the record addresses. If mail fails it is logged and swallowed — it never undoes the custody change, so *confirm the recipient actually received it* rather than trusting the success screen.

You will notice the builder may warn "Held by X, not Y" if the device already shows a different holder. **That is a warning only — nothing stops you issuing a device that is already out on another open receipt.** See backlog item B1.

Returning equipment

Returns are **admin-only**. Open the receipt (`/receipts/<number>`) and use the return flow. Tick the serials coming back and sign. A **partial** return records the handback and leaves the receipt open for the rest; a **full** return closes it. Once closed a receipt is **immutable** — it cannot be reopened, edited, or returned against again — and it is permanently deleted 90 days after closing.

An admin can also put a **return deadline** on an open receipt from the receipt page. A nightly job emails one overdue alert per lapsed deadline.

Working the service queue

`/admin/queue` lists every item whose service entry is `PENDING`, showing serial, device name, unit, service type and actions. You can search, filter by service type, sort, and show/hide columns.

- **Mark Completed** takes an item off the active queue. It is *not* a delete — the record is kept and can be reopened to `PENDING` from the item page. Completing a service item also stamps the device as on-hand, because it is physically in front of you at that moment.
- A **deadline** is optional and comes only from a number of days someone typed. **Blank means no deadline, and blank on an edit means "leave the existing deadline alone."** To actually clear a deadline you must use the item page's own "Update deadline" control — that is the only place clearing is expressible, deliberately, so a stray blank field can never silently wipe a date.
- Re-flagging or reopening a completed item starts a **new round** and resets its deadline and alert stamp; editing a live request's note or type does not touch the deadline.

Running a CSV / MDM import

`/admin/items/import` is a two-step, admin-only flow:

1. Choose the export and click **Analyze**. This **writes nothing** — it runs the same parse and the same reads the real import does and shows the counts it would apply, plus any rows whose home unit it could not decode.
2. Resolve unrecognised units by hand if you want to (you do not have to — an unresolved unit imports the row with a blank home unit and reports it, it never blocks the row).
3. Click **Import**.

Things to know before you rely on it:

- `serialNumber` **is the match key.** An existing serial is **updated in place**; a new one is created. **Nothing is ever deleted** — a device missing from the export is left alone, not retired. Absence from the CSV is not a signal.
- **The CSV overwrites.** For a device that already exists, the export is treated as the source of truth for its device name, home unit, category and assigned user — including overwriting a hand edit someone made in the app since the last import. Every field the import changes is recorded in that item's edit history attributed to `MDM Import (automated)`, so you can always see what a run touched.
- **Maximum 2,000 rows per file.** Split a larger export.
- The category comes from a `deviceType` column (also `deviceCategory` / `category`). A bare `type` column is deliberately ignored, because MDM exports use it for OS strings.
- **A failed import writes nothing.** The whole run is one transaction, so there is no partial state to reconcile — on a failure, re-run it.
- During the import the only feedback is a disabled button reading "Importing…". There is no progress bar today; see backlog item B3.

There is also a machine door, `POST /api/items/import`, authenticated with a bearer `MDM_IMPORT_SECRET`, for a scheduled nightly export. `DEPLOY.md` §7 and §7a document how to prove it works in production before handing it to a scheduler.

Auditing an item

Open a device at `/i/<id>`. The **Audit** card shows a coloured light: compliant, overdue, or never audited. Items are audited **annually**. An admin sees an **Audit** control there — recording an audit snapshots your name and signature and updates the light immediately. The card also shows the full audit history.

Note that the audit is what proves **possession**. There is no tick-box for "we have this" — the system only claims accountability where an audit says so.

The admin surfaces

Everything admin-only hangs off `/admin` (the **Dashboard** link in the header), which shows overdue and due-soon hand receipts and service items, plus a **Manage** row linking to:

Where	What you do there
`/admin/analytics`	Readiness analytics — audit readiness donut, fleet KPIs by category, DA 2062 volume, unit leaderboard. Filter the whole page by unit or UIC; export any chart to PNG/CSV or switch it to a table.
`/admin/categories`	The curated list of device classes. Deleting one is refused while items still carry it.
`/admin/units`	Unit abbreviation → full name, bulk-pasteable. **Renaming a unit rewrites every item holding the old spelling** — but the next import can overwrite it again, so the "N items updated" message describes that moment, not a permanent guarantee.
`/admin/queue`	The service queue (above).
`/admin/users`	Create accounts, set role, activate/deactivate. You cannot demote or deactivate yourself. Also the shared contact address book.
`/admin/audit`	The full edit/audit log.
`/admin/items/new`	Log a new device by hand.
`/admin/items/<id>/edit`	Full field edit. Note **make / model / serial number have their own separate card** — correcting a serial is deliberately a distinct act. **Correcting a serial does NOT change existing receipts**, which keep the serial they were signed with, permanently. That is correct behaviour for a signed document; the form says so.
`/admin` (bottom)	Set or rotate the **public access PIN**. Rotating stops new unlocks immediately, but visitors already unlocked stay in for up to 12 hours.
`/admin/items/qr-sheet`	Print a sheet of QR labels for a selection.

2.2 For a developer

Prerequisites: **Node ≥ 20** and **Docker**.

```
# 1. Install
npm install

# 2. Start Postgres (docker-compose.yml → postgres:16 on host port 5435)
docker compose up -d

# 3. Configure env
cp .env.example .env
#    AUTH_SECRET:   npx auth secret
#    DATABASE_URL / APP_URL already point at the local DB and localhost:3000

# 4. Migrate + seed an admin
npm run db:migrate
npm run db:seed        # requires SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD — prisma/seed.ts
                       # throws without them; there is no built-in default account.

# 5. Run
npm run dev            # http://localhost:3000
```

The integration suite needs a **second database on the same server**, created once:

```
CREATE DATABASE handreceipt_test;
```

Command	What it does
`npm run dev`	Dev server (Turbopack), after staging the wasm assets (`scripts/copy-wasm.mjs`).
`npm run build`	`copy-wasm && prisma generate && next build`.
`npm test`	Full Vitest suite against the real migrated `handreceipt_test` DB.
`npm run test:ui`	Only the jsdom component tests (`*.test.tsx`).
`npm run lint`	ESLint 9.
`npx playwright test`	E2E. Seed first with `npm run db:seed:e2e`.
`npm run check:security-docs`	Runs the `Security docs current` CI gate locally.
`npm run db:migrate` / `db:deploy` / `db:reset`	`prisma migrate dev` / `deploy` / `reset --force`.
`npm run db:seed:analytics`	**Dev only.** Populates demo analytics data; refuses any non-local `DATABASE_URL`.

Four things that will bite you in the first week:

1. **Only one person or agent may run the test suite at a time.** It truncates a shared `handreceipt_test` database; two concurrent runs corrupt each other and the failures look like unrelated flakes in files you did not touch.
2. **`prisma migrate dev` may not run in every shell here.** The project's convention when it cannot is to author migrations with `prisma migrate diff --from-config-datasource --to-schema --script` and then `migrate deploy`. Prisma 7 rejects the older `--from-schema-datasource` / `--to-schema-datamodel` flags.

3. **Neither `npm run build` nor jsdom is evidence for a CSS or mobile change.** Neither has a layout engine. Verify visual work in a real browser.
4. **Work on a branch.** `main` is protected and needs a PR with three green checks.

---

3. Technical architecture

3.1 Stack — verified versions from `package.json`

Layer	Technology	Version (`package.json`)
Framework	Next.js (App Router, Server Components, Server Actions, Route Handlers)	`16.2.9`
UI runtime	React / React DOM	`19.2.4`
Language	TypeScript	`^5`
Bundler	Turbopack	via `next dev` / `next build`
ORM	Prisma Client	`^7.8.0`
DB driver adapter	`@prisma/adapter-pg` over `pg`	`^7.8.0` / `^8.22.0`
Database	PostgreSQL — Supabase (prod), Docker `postgres:16` (local)	`postgres:16` in `docker-compose.yml`
Auth	Auth.js v5 (`next-auth`), Credentials + JWT	`^5.0.0-beta.31`
Password hashing	`bcryptjs` (cost 12)	`^3.0.3`
Validation	Zod	`^4.4.3`
Rate limiting	`@upstash/ratelimit` + `@upstash/redis`	`^2.0.8` / `^1.38.0`
PDFs	`pdf-lib`	`^1.17.1`
QR generation / scanning	`qrcode` / `barcode-detector`	`^1.5.4` / `^3.2.1`
CSV	`csv-parse`	`^7.0.1`
Charts	`recharts`	`^3.10.1`
Icons	`lucide-react`	`^1.27.0`
PNG export	`html-to-image`	`^1.11.13`
Styling (new UI)	Tailwind CSS v4 + shadcn/ui primitives over Radix	`tailwindcss ^4.3.3` , `@radix-ui/*`
Testing	Vitest / Playwright / Testing Library	`^4.1.9` / `^1.61.1` / `^16.3.2`
Linting	ESLint + `eslint-config-next`	`^9` / `16.2.9`

There is **no SMTP client**. `nodemailer` was removed on 2026-08-04. Outbound mail is a plain `fetch` to the Gmail API, or to Resend, or a logging stub.

3.2 App Router structure

```
src/
  app/
    page.tsx                    public home + search (NOT PIN-gated)
    actions/                    user-facing Server Actions (auth, receipts, returns,
                                items, search, scan, contacts, signatures, account, unlock)
    admin/
      actions/                  admin-only Server Actions (items, users, queue, units,
                                categories, readiness, audit, contacts, receipt-timer,
                                public-access, verify-seal)
      analytics/ audit/ categories/ items/ queue/ units/ users/
      dashboard/dashboard.service.ts
      page.tsx                  the admin hub (timers + Manage links + PIN control)
    account/                    password change + saved signatures
    i/[itemId]/                 public item page, + /qr/pdf route
    items/                      signed-in inventory list
    receipts/
      new/                      the hand-receipt builder
      [receiptNumber]/          public receipt page, /pdf route, /return flow
    login/ forgot-password/ reset-password/ unlock/ privacy/ terms/
    api/auth/[...nextauth]/     Auth.js route handlers
    api/cron/purge/             nightly worker (CRON_SECRET)
    api/items/import/           machine CSV import (MDM_IMPORT_SECRET)
  components/                   shared UI, incl. components/ui/ (shadcn primitives)
  lib/                          prisma, authz, rate-limit, crypto, email, datetime, …
  modules/                      domain services: items, transfers, receipts, returns,
                                service-queue, audit, users, contacts, signatures,
                                timers, auth
  auth.ts                       Auth.js configuration
  proxy.ts                      Next 16 proxy (was: middleware) — Node runtime
```

Server Components are the default. Pages fetch through `modules/*` services directly. Client Components (`"use client"`) exist only where interaction requires them — the receipt builder, the signature pad, the QR scanner, the item selection table, the combobox inputs, the chart cards.

Server Actions vs Route Handlers — the rule the codebase follows:

- **Server Actions** back every interactive form. They *limit themselves* (rate limiting lives inside the action) and return `{ error }` objects rather than throwing, because a 429 or a throw from an action POST cannot be rendered by `useActionState` and escalates to the error boundary.
- **Route Handlers** exist only where something other than a React form is the caller: the Auth.js endpoints, the PDF downloads, the QR sheet, the cron worker, and the machine CSV import. The last two authenticate with a constant-time bearer-secret compare because they have no session.

There are **49 Server Actions across 22 files and 6 Route Handlers**, and per `SECURITY_ASSESSMENT.md:228-230` every one of them gates itself as its first awaited statement, and all 10 admin pages call `requireAdmin()` directly rather than inheriting a gate from a layout.

3.3 The authorization model — role-based, explicitly NOT ownership-based

This is the single most important thing to internalize, because the instinct it contradicts is a strong one.

Inventory, receipts and the service queue are shared org-wide. Do **not** add `session.user.id` ownership filters to item, receipt, or queue queries. A technician is not supposed to see "their" items; they are supposed to see the property book. Gate on the **role**, never on "the user happens to own no rows" — a demoted admin keeps their rows.

Every Server Action and Route Handler starts with `requireUser()` or `requireAdmin()` from `src/lib/authz.ts` — **never a bare** `auth()`. The reason is at `src/lib/authz.ts:27-38`:

```
const defaultGetSession: GetSession = async () => {
  const { auth } = await import("@/auth");
  const session = await auth();
  if (!session?.user) return null;
  const { default: prisma } = await import("@/lib/prisma");
  const fresh = await prisma.user.findUnique({
    where: { id: session.user.id },
    select: { role: true, isActive: true },
  });
  if (!fresh || !fresh.isActive) return null;
  return { user: { ...session.user, role: fresh.role } };
};
```

The JWT carries the role captured at login. This re-reads `role` and `isActive` **from the database on every protected request**, so a demotion or deactivation takes effect on the user's next request rather than whenever their token happens to expire. It costs one indexed `findUnique` per request, which is a deliberate and documented trade (`docs/SECURITY.md` Known gap 5).

`requireAdmin()` (`src/lib/authz.ts:48-54`) is required for: returns, user management, named signatures, service-queue mutations, receipt and service timers, audits, imports, categories, units, readiness edits, item create/edit/delete, and analytics.

A standard `USER` may read the shared inventory, create receipts, and edit **only** `currentUserEmail` + `currentPosition` (`userItemDetailsSchema`). This is enforced by `updateItemDetailsAction` picking the Zod schema **by role**, so the restriction is server-side; `z.object()` strips undeclared keys, which is both the mechanism and a trap (see §3.11).

Session lifetime is 10 hours absolute / 4 hours idle, policy in `src/lib/session-freshness.ts`, enforced in the `jwt` callback of `src/auth.ts` — not in the proxy, because the callback runs on every `auth()` call including Server Actions and RSC.

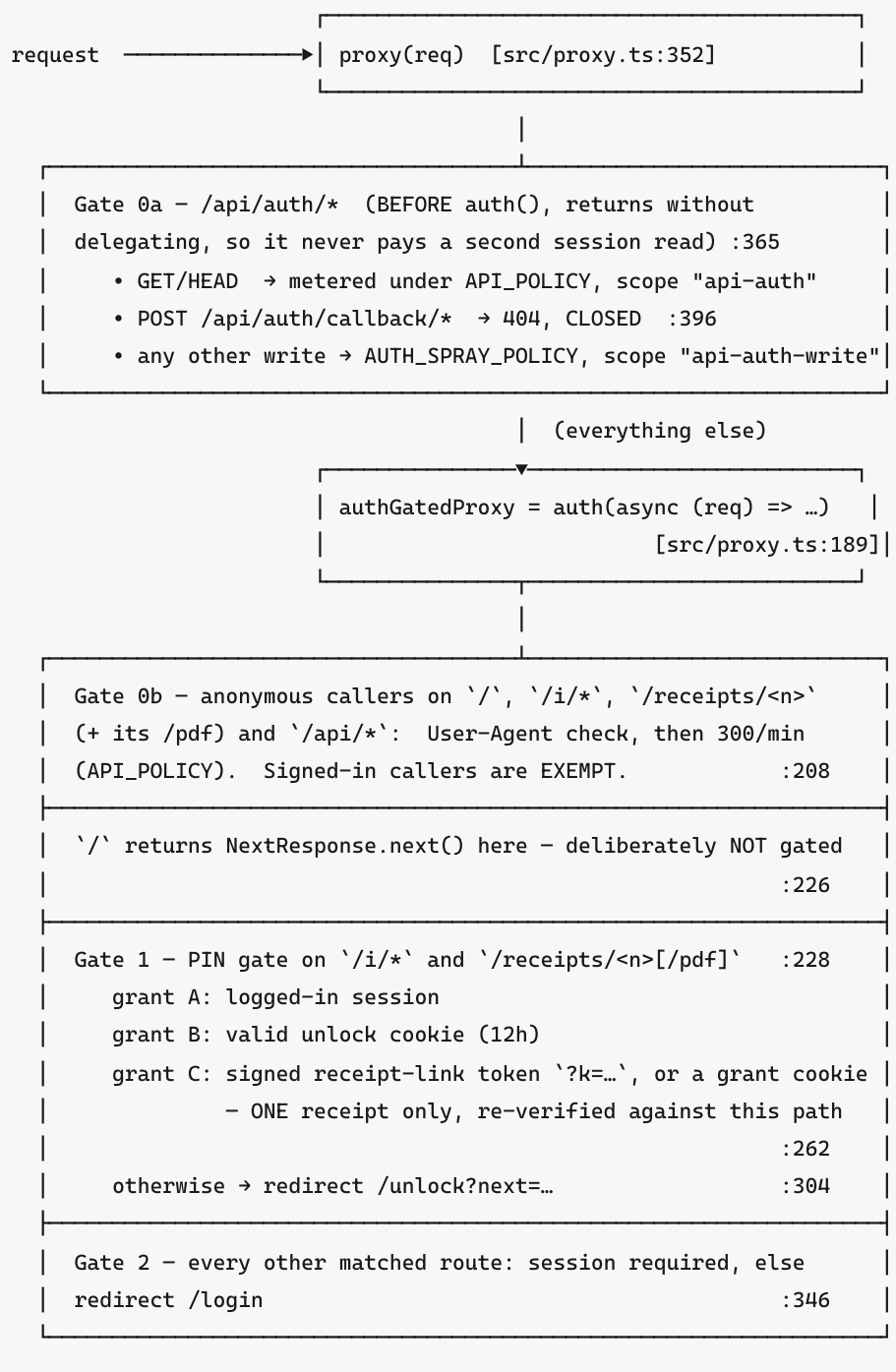
The absolute bound is an `authAt` claim stamped once at sign-in and never moved; `lastActiveAt` is the one that moves. **`session.maxAge` alone is not an absolute expiry** — Auth.js re-signs the token on every `auth()` call, so it behaves as an idle timeout.

The only session **revocation** lever is `User.passwordChangedAt`, compared in the `jwt` callback (`src/auth.ts:106-137`). Since 2026-08-05 all three password-mutation paths stamp it: admin-initiated reset (`src/lib/password-reset.ts:46`), deactivation (`src/modules/users/users.service.ts:47`), and — newly — self-service change. Note it is stamped only on the *success* path; stamping on a wrong-current-password refusal would let anyone who can reach the form log the real owner out. Two tests in `users.service.test.ts` pin both halves.

3.4 `src/proxy.ts` — the layered gates

In Next 16 the file formerly known as middleware is `proxy.ts`, and Next allows a single `proxy` export. This one carries **three gates** in a deliberate order, and it runs on the **Node.js runtime**, which is what lets it import `@/auth` (and therefore Prisma).

It is not the authorization boundary. Real authz stays per-route in `requireUser` / `requireAdmin`. The proxy is a coarse redirect gate plus anti-abuse.



Details that are load-bearing and easy to break:

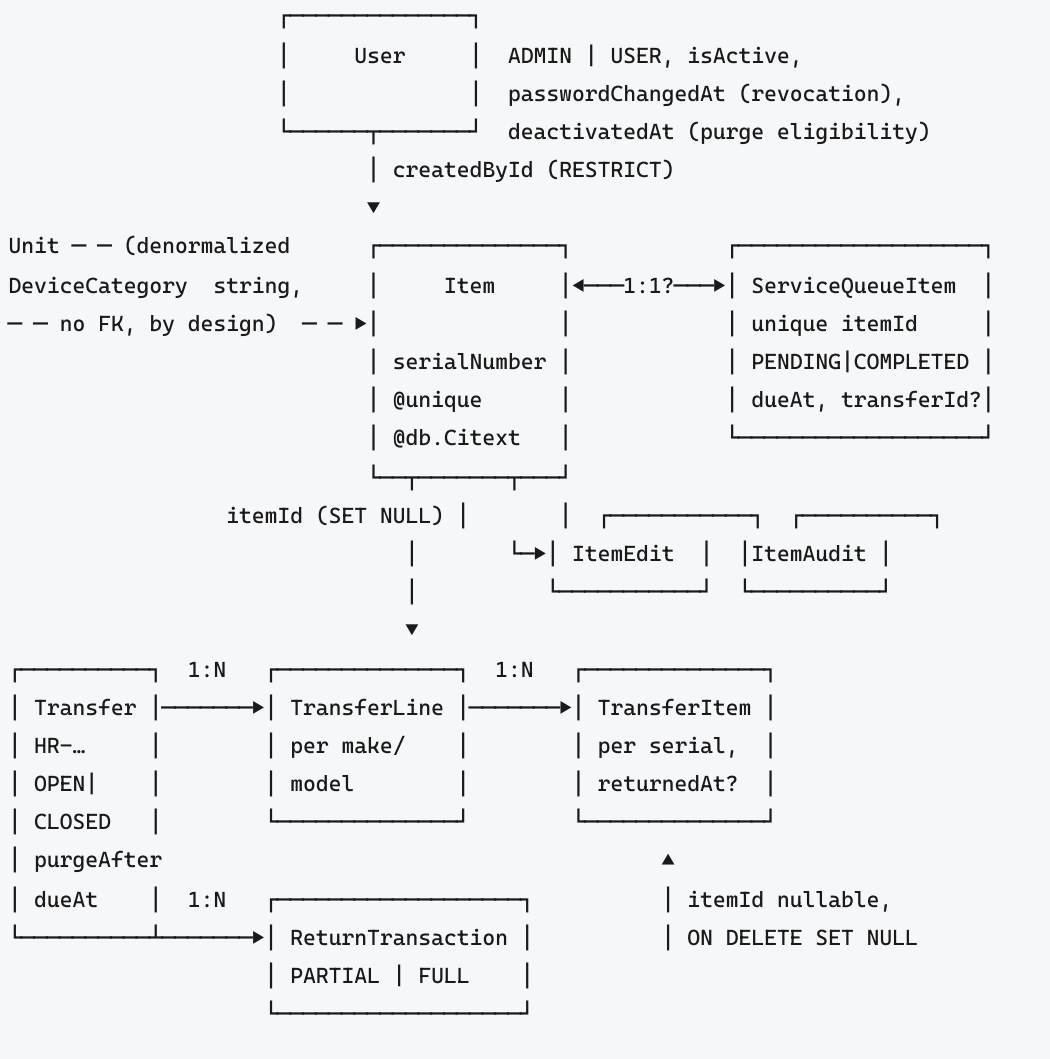
- **The order of grants inside gate 1 is deliberate** (`src/proxy.ts:249-261`). The receipt-link branch runs *after* the session and unlock-cookie decision, so a technician or an already-unlocked visitor is never narrowed down to a single receipt by clicking a link in their inbox.
- **The receipt token deliberately does not widen `publicAccessAllowed()`.** The proxy and `src/lib/public-access-guard.ts` now admit different populations on purpose, and `src/lib/public-access-guard.test.ts` pins that the guard refuses a receipt grant. The token opens `/receipts/<n>` and `/receipts/<n>/pdf` and nothing else; `/i/*` yields no receipt number at all, so the branch structurally cannot reach the device catalog.

- `/` **is outside the PIN gate**, and the gate it used to inherit moved onto the search action. `liveSearchAction` (`src/app/actions/search.ts:26`) calls `publicAccessAllowed()` itself, **before** the query, and **that call is the entire gate on the public search** — not defence in depth. A refusal returns `{ locked: true }`, never an empty result, because "No matches" would be a confident wrong answer about the property book.
- **The matcher exclusions are segment-anchored** with `(?:/|$)` (`src/proxy.ts:466`). Unanchored they were prefixes, and `/api/cronjobs` , `/logins` or `/unlockables` would have skipped the proxy entirely.
- `api/cron` **and** `api/items/import` **are excluded from the matcher on purpose** — both authenticate with a constant-time secret compare and have no session cookie. Without the exclusion, PowerShell's `Invoke-RestMethod` would follow the 302 to `/login` and report a 200 while importing nothing.
- **The User-Agent filter is not a control** (`src/proxy.ts:154-177`). One line defeats it. Nothing may depend on it, and it stays anonymous-only because a signed-in technician may legitimately be driving a script. `HeadlessChrome` is deliberately **not** on the list — adding it broke this repo's own Playwright suite.

Rate limiting lives in one module, `src/lib/rate-limit.ts` : `AUTH_POLICY` (5/15 min), `AUTH_SPRAY_POLICY` (60/15 min), `UNLOCK_POLICY` (20/15 min), `API_POLICY` (300/min anonymous). Store is Upstash Redis via the `KV_REST_API_*` pair; unset falls back to per-instance counters, which is real but not the production posture. It **fails open** on a store error, deliberately and documented (`docs/SECURITY.md` §12, Known gap 1).

3.5 Data model

The core is three tables. `prisma/schema.prisma` is heavily commented and is the right place to read the detail; what follows is the shape and the relationships.



Entity notes that change how you should write queries:

- **`Item.serialNumber` is `@unique @db.Citext`** (`prisma/schema.prisma:81`) — a device's case-insensitive identity, like `User.email`, `Unit.abbreviation`, `DeviceCategory.name` and `Contact.email`. Never assume case-sensitive distinctness. Consequence: a text `gin_trgm_ops` index does **not** serve citext's own ILIKE operator, so the serial search must cast `"serialNumber"::text ILIKE …` to actually use `Item_serialNumber_trgm_idx` — see `searchItemsBySerial`, `src/modules/items/items.service.ts:775`.
- **There is no `currentHolderId` on `Item`.** Who holds it now is derived, and there are **two deliberately different rules**:
  - `getHoldingTransfer` / `getLastReceiver` (`src/modules/transfers/transfers.service.ts:148-175`) take the item's **latest** receipt and **fail closed** — no row, a non-OPEN receipt, or any `returnedAt` on this item's rows names nobody. Its answer prefills a signed DA 2062, where naming the wrong holder is worse than naming none.

  - The list / search / readiness surfaces use the looser shared predicate in `src/modules/transfers/custody.sql.ts:44` — `ti."returnedAt" IS NULL AND t."status" = 'OPEN'` over **any** open receipt.

  These two can disagree when an item sits on an older open receipt and a newer closed one. **Do not unify them** without deciding what the receipt builder should get.

- **`TransferItem.itemId` is nullable, `ON DELETE SET NULL`** (`prisma/schema.prisma:342`). Deleting an item *detaches* its receipt lines rather than erasing them. Every field a receipt renders — serial on `TransferItem`, make/model on `TransferLine`, signatures on `Transfer` — is a **snapshot written at creation and never joined back to `Item`**. That is why correcting an item's serial number leaves every existing receipt showing the old one, permanently. It is the right behaviour for a signed document.
- **Both parties on a `Transfer` are typed snapshots, not FKs to `User`.** A receipt fully describes who gave and received an item even if no account for either ever existed.
- **`Unit` and `DeviceCategory` are managed vocabularies with denormalized values, not foreign keys** — because a CSV import must be able to carry a unit or category the property book has not registered yet, and an unknown value must never fail an import. Coherence is enforced in the service layer instead, two ways: **deletion is refused while any item still carries the value**, and **imports register unseen values** (`learnUnits`, `learnCategories`). Do not "normalise" this into an FK without re-solving both.

3.6 Derived state — and why it is not a column

This is the design decision a future maintainer is most likely to try to "fix". Read this section before touching readiness or accountability.

Two derived states

Readiness — "can this go out" — is computed by `readinessState()` in `src/modules/items/readiness.ts:101-108`:

```
if (s.status === "RETIRED") return "RETIRED";
if (s.flaggedForService) return "IN_REPAIR";
if (s.onOpenReceipt) return "DEPLOYED";
if (s.markedReadyAt) return "READY_TO_DEPLOY";
if (s.lastLogonUser && s.lastLogonUser.trim() !== "") return "DEPLOYED";
return "UNTRIAGED";
```

The precedence is the design. The service flag is checked *before* the receipt rule, which is what lets the receipt rule exist at all — a device turned in for repair while its receipt is still open must not read "Deployed". The MDM-logon rule is last because it is the weakest and stickiest evidence; it exists to cover the roughly 1,053 devices genuinely in soldiers' hands that predate the app and carry no hand receipt.

A `DEPLOYED` rule reading `lastLogonAt > markedReadyAt` used to sit between the receipt rule and `READY_TO_DEPLOY`, making the hand-set marking self-expire. It was **removed from both twins**: a device on our own shelf produces logons routinely (imaging it, an MDM check-in, a

test before reissue), so it read "Deployed" while physically in hand, contradicting the person who had just held it. `lastLogonAt` is still parsed and stored, so reinstating it is a one-line change — **in both twins**, or the parity test fails.

Accountability — "do we physically have it" — is derived from audit recency. `auditState(lastAuditedAt, now)` (`src/modules/audit/audit.status.ts:35-40`) returns `"never"` for null, else `"compliant"` if `now < lastAuditedAt + AUDIT_PERIOD_YEARS` (which is `1` , `:7`), else `"overdue"` . The SQL side shares `auditCutoff(now)` (`:57-61`), so the badge and the dashboard cannot disagree about the boundary.

Why these are not stored columns

Because **stored versions of both were shipped, measured, and dropped.**

- `Item.isAccountedFor` was `BOOLEAN NOT NULL DEFAULT true` , written by almost nothing. Production held **1,139 items with zero marked not-accounted-for**, so the column reported a fully accounted-for fleet out of its own default. Dropped in `20260727234500_drop_is_accounted_for` .
- `Item.deployableStatus` (a stored enum) plus its `ItemStatusHistory` table were written only by a bulk admin action: **0 of 1,139 rows ever had a value**, so every device read "Untriaged". Dropped in `20260728000000_derive_readiness` .

The lesson, written into `prisma/schema.prisma:118-134` : a stored status column measures its own default, not the fleet. **Possession is claimed only where an audit proves it; readiness is claimed only where a live signal shows it.** Do not reintroduce either.

The one hand-set signal is `Item.markedReadyAt` — a **timestamp**, not a boolean, so the marking can be *compared* to other dated signals and so "when did we last have hands on this" is answerable. It persists until a deliberate act supersedes it: an open receipt, a service flag, retirement, or an explicit clear.

The twin, and the test that keeps them honest

Readiness is written **twice**: once in TypeScript for one row (`src/modules/items/readiness.ts`), once in SQL for the whole fleet (`src/modules/items/readiness.sql.ts` — `READINESS_CASE` , `READINESS_JOINS` , `READINESS_RANK`). The SQL twin exists because the dashboard and the `/items` page must bucket and order 1,100+ items *in the database*, not in JavaScript.

`src/modules/items/readiness.parity.test.ts` runs one fixture table through **both** and asserts they agree on every branch and precedence boundary. Change the precedence in one and that test fails until you change the other. **Never add a third derivation** — a surface needing readiness calls `readinessForItems` (`readiness.query.ts`) or embeds the shared SQL fragments.

Sorting a derived value: the second query path

Because readiness is derived from four signals across three tables, there is no column for a Prisma `orderBy` to name. So `listItems` (`src/modules/items/items.service.ts`) **branches**:

- A sort whose keys include a **derived key** selects the ordered, paginated page of item **ids** with a parameterized `$queryRaw` (`derivedOrderedItemIds` , `items.service.ts:262`)

joining `READINESS_JOINS` and ordering by `READINESS_RANK`, then hydrates them with `getItemsByIds`. Two bounded queries, no per-row derivation.

- **Every other sort stays on the untouched Prisma path.**
- The row count always comes from the one Prisma `count` (`items.service.ts:390`).

`auditState` is the **other** derived key and takes the same raw path. `DERIVED_SORT_KEYS` (`src/modules/items/sort-keys.ts:60-63`) is the set, and `columnForKey` (`items.service.ts:91-96`) **refuses** both with a typed error, so a caller that forgets to branch gets an exception rather than a bad `ORDER BY`. It used to map to `lastAuditedAt`, which sorted by raw *recency* while the column displays a three-value *badge* — and because a timestamp is nearly unique per row (production: 31 audited rows, 31 distinct stamps), a secondary sort key had no ties to break and silently did nothing. It is now ranked by `auditRankSql` (`src/modules/audit/audit.sql.ts:58-61`) over `AUDIT_ORDER` — the same array the analytics donut stacks by.

Because both paths implement the `?q=` / `?uic=` filter, two filter implementations can drift. `src/modules/items/items.readiness-sort.parity.test.ts` seeds real rows and asserts both return the **same ids in the same order and the same total** for the same filters. This test is the guardrail for backlog item B8.

Sort keys are spliced into SQL as **identifiers**, so they come from the frozen `SORT_COLUMN` allowlist (`sort-keys.ts:26-34`), never from the querystring, and the lookup uses `Object.hasOwn` specifically so `"toString"` cannot return an inherited function.

3.7 Styling — two systems, on purpose

Summarized here; **the full rationale is in `CLAUDE.md`** and in the 50-line header comment of `src/app/styles.css`, and you should read both before touching CSS.

- **`src/app/globals.css` is the original design system** — the "property book" ledger look (`.card`, `.stack`, `.btn`, `.table`) backing every pre-existing page. It is **not** being migrated.
- **Tailwind v4 + shadcn/ui are for NEW UI only** (`src/components/ui/*`, the analytics dashboard). Do not rewrite existing pages to Tailwind as a drive-by.
- `src/app/styles.css` makes the coexistence safe, via three rules that must not be "simplified":
  1. **Preflight is deliberately not imported** (`styles.css:54-58`) — only Tailwind's `theme` and `utilities` layers. Importing `tailwindcss` wholesale pulls in preflight and restyles every existing page.
  2. **`globals.css` is imported into a `legacy` cascade layer declared before Tailwind's layers** (`styles.css:51-53`). Unlayered CSS outranks *all* layered CSS, so without this every Tailwind utility would silently lose to `globals.css`.
  3. **`@source` is narrowed to two directories AND `@source not inline("container")` blocks that one candidate outright** (`styles.css:66-89`). Narrowing alone proved insufficient — the extractor reads raw file *text*, so it harvested `container` from `data-slot="table-container"` and from the English word in `table.tsx`'s prose comments,

generating a `.container` utility that widened every pre-existing page from 720px to 1280px at a 1440px viewport. **Do not delete that exclusion.**

Because preflight is absent, shadcn primitives must supply what it normally does: `appearance-none` plus an explicit background/font on `<button>`-rooted parts, `border-solid` **wherever a** `border-*` **width is set**, and `[&_svg]:block` on icons. And the corollary that already shipped a bug: **a single-side border needs the other three sides zeroed explicitly** — write `border-x-0 border-t-0 border-b border-solid border-border`, or you get a 3px box instead of a 1px underline.

shadcn tokens are mapped onto the ledger palette via `var()` references, so retuning `globals.css` retunes the dashboard. shadcn's `--primary` is aliased to `--primary-token` because `globals.css` already defines `--primary`.

What is actually in `src/components/ui/`

Seven primitives, and **only** these. There is no component library to browse — if you need something else you are adding it, with the checklist below. Reuse before you add.

Primitive	Used by	Notes
`button.tsx`	Analytics dashboard, newer controls	Size variants `sm` / `default` / `icon`, all with tap-target overrides
`card.tsx`	Analytics widgets	Ledger-styled container; not the legacy `.card` class
`dropdown-menu.tsx`	Column pickers, row actions	Radix-backed
`select.tsx`	Analytics filters	Trigger **and** item both carry tap overrides
`table.tsx`	Analytics tables	The single-side-border bug shipped here — see the corollary above
`toggle-group.tsx`	Chart range / view toggles	
`progress.tsx`	Search progress indicator	Newest; its header comment records that it sets no `border-*` width, so the `border-solid` trap does not apply. Keep it that way

Everything else under `src/components/` is app-specific rather than a primitive — `SuggestCombobox` (the mobile-working suggestion box used on every item-edit surface), `ItemSelectTable`, `DeleteItemButton` (the reference implementation for the `<dialog>` rule in the landmines table), `HomeSearch`, `TurnstileWidget`.

Touch targets — a 44px floor that shadcn does not give you

`globals.css:61-62` sets `--tap: 44px` and `--tap-lg: 48px`. **shadcn's stock sizes are below it** — `h-8` is 32px and `h-9` is 36px — so every interactive primitive carries an explicit restore: `pointer-coarse:h-11 max-md:h-11` (`h-11` = 44px), and `size-11` for icon buttons. Verified present in `button.tsx:45-48`, `select.tsx:41` and `:163`, and `toggle-group.tsx:34-35`.

The consequence is easy to miss: **override a height and you silently drop through the floor.** Setting `h-auto` or a custom height on one of these, or writing a new interactive component without the two variants, ships a ~22–36px tap target on a phone. Restore the floor explicitly whenever you touch a height.

Adding a new shadcn primitive — the checklist

Copying a component in from shadcn upstream **will not work unmodified**, because upstream assumes preflight is loaded and it is not. Before you commit one:

1. `appearance-none` **plus an explicit background and font** on any `<button>`-rooted part (present today in `button`, `select`, `dropdown-menu`, `toggle-group` — the four that have one).
2. `border-solid` **wherever any** `border-*` **width is set** — Tailwind's border utilities set width only. Present in all seven primitives.
3. **A single-side border needs the other three zeroed**: `border-x-0 border-t-0 border-b border-solid border-border`. Skipping this is what put a 3px frame on the analytics tables.
4. `[&_svg]:block` on any icon slot.
5. `pointer-coarse:` **and** `max-md:` **height variants** to hold the 44px floor.
6. **Do not fix any of the above with a global reset** — a base `border-width: 0` or a preflight re-import is precisely what §3.7's first rule exists to prevent.
7. **Verify it in a real browser at a phone viewport.** Neither `npm run build` nor `npm run test:ui` has a layout engine, so neither can see any of this.

3.8 Background jobs, email, PDFs and QR

The nightly worker

One endpoint, `GET|POST /api/cron/purge` (`src/app/api/cron/purge/route.ts`), does four things in a single run, all in parallel:

Sweep	Behaviour
`purgeExpiredTransfers`	Hard-deletes closed receipts past `purgeAfter` (= `closedAt` + 90 days).
`purgeDeactivatedUsers`	Hard-deletes accounts deactivated 3+ months ago — but **skips** any still referenced by items or import batches (`ON DELETE RESTRICT`), reporting them as `skippedCount`.
`sendOverdueTransferAlerts`	One email per lapsed receipt deadline, stamped so it never re-alerts.
`sendOverdueServiceAlerts`	The same, for pending service items.

It authenticates with a **constant-time** `CRON_SECRET` **bearer compare and fails closed** (`route.ts:20-22`). Unset means every call is a 401 and **nothing is ever purged or alerted**.

It runs from GitHub Actions, not Vercel Cron — `.github/workflows/purge-cron.yml`, daily at 08:23 UTC, and the workflow hard-fails on any non-200 or a body without `"ok":true` (`:28-29`) so a broken secret is visible in the Actions tab. There is deliberately **no** `vercel.json` **and no Vercel Cron**: on the Hobby plan the schedule never fired and the purge silently never ran. Do not "restore" it.

Email

Transport is a pluggable `EmailSender` (`src/lib/email.ts`) selected **by environment presence only** — never by falling back on a send failure, because an expired credential would then silently reroute mail instead of surfacing:

1. **Gmail API** (OAuth2 refresh token, scope `gmail.send`) when all four `GMAIL_*` vars are set;
2. **Resend** over `fetch` when `RESEND_API_KEY` + `EMAIL_FROM` are;
3. otherwise a **logging stub** (`[email:stub]`).

Three custody flows send mail — new receipt, return, pickup — and all three address themselves through one module, `src/lib/email-recipients.ts` (`addressCustodyEmail`), so "who saw this?" has a single answer. Each sends **one** message: the non-DCSIM customer on `To` , the record copies (`RECEIPT_CC_EMAILS` , which **defaults to real addresses when unset** — empty and unset mean different things) and `ADMIN_INBOX_EMAIL` on `Cc` . Send failures are logged and swallowed; they never roll back a created receipt.

Two operational facts that cost real time to discover:

- **Any link in a custody email must be built from `defaultBaseUrl()` (`src/lib/base-url.ts`), never a hardcoded deploy URL.** A `vercel.app` URL *in the message body* was enough to make the whole message vanish for `army.mil` recipients with no bounce — proven by a controlled four-message test. `APP_URL` must be `https://www.dcsim.us` .
- **The Gmail consent screen is kept in *Testing* status**, so Google expires the grant every ~7 days and `GMAIL_REFRESH_TOKEN` must be re-minted and pushed to Vercel **plus a redeploy**. `scripts/gmail-token-rotation/` automates this on a Windows workstation. Note its side effect, recorded as accepted risk A10: that tool fires a Deploy Hook and ships **whatever is on `main` at that moment, with nobody watching** — which interacts badly with the migrate-before-merge rule.

PDFs

`src/modules/receipts/hand-receipt.ts` (`buildHandReceiptPdf` , `:58`) fills the official DA 2062 AcroForm with `pdf-lib` , flattens it, draws the recipient's signature **rotated 90°** in the quantity column with the date, paints black anti-tamper guard bars above and below it (`:172-179` — the subject of backlog item B2), embeds a QR code of the receipt URL, and appends a custody-record page. The template is embedded as base64 so it bundles reliably on serverless.

`src/modules/receipts/render.ts` is the data-assembly layer above it and the sole caller. It owns `quantityColumns` (`:12-24` , the A–F numeric series, capped at 6) and the rule that **only PARTIAL returns get a signature column B–F** (`:52-56`) — the closing FULL return renders as a `CLOSED` watermark plus an "accepted by" attestation instead. It also injects the PIN-bypass receipt link into the QR (`:70`).

The route is `/receipts/[receiptNumber]/pdf` — **public, no login required**, since anyone with the receipt number or the QR link should be able to pull the PDF.

`src/modules/items/qr-sheet.ts` builds printable label sheets (`buildItemsQrSheetPdf`), using only `drawImage` and `drawText` ; the layout maths lives in `qr-sheet-layout.ts` .

QR

`src/modules/items/qr.ts` generates QR data URLs and **caches them across requests and deploys** with `unstable_cache`, keyed on the resolved URL — they are immutable, so they are never re-encoded per request.

One nuance worth carrying forward: a QR code is a picture of a URL, so labels printed before the 2026-08-04 domain change encode `servicedeskapp.vercel.app`. **The app's own scanner still works on them** (it reads only the item path and ignores the origin), but a plain phone camera on a government network will not open them.

3.9 CI/CD, branch protection, and migrate-before-push

`main` is **branch-protected**. Merging requires a PR whose **three** required checks pass, all defined in `.github/workflows/ci.yml`:

Check	Job	When	What it does
`Semgrep SAST`	`sast`	push + PR	Runs Semgrep from the official `semgrep/semgrep:1.90.0` **docker image** via `docker run`. SARIF upload is informational; the blocking gate fails only on ERROR severity.
`Build (next build)`	`build`	push + PR	`npm ci && npm run build` with dummy env values.
`Security docs current`	`security-docs`	**PR only**	`node scripts/check-security-docs.mjs "origin/$BASE_REF"` — fails a PR that touches a watched security file without touching `docs/SECURITY.md`.

`strict` is on (the branch must be up to date with `main`). Admins may bypass in an emergency (`enforce_admins: false`), but the default path is **branch → PR → green → merge**, and Vercel deploys production from `main` on merge.

Two things about that setup:

- **Semgrep must stay on the docker image.** A host `pipx` install crashes on the runner's Python 3.12 with `ModuleNotFoundError: pkg_resources` (setuptools ≥ 81 removed it).
- **`github.base_ref` goes through an intermediate env var** rather than `${{ }}` interpolation in `run:` (`ci.yml:95-97`), because a branch name is attacker-controlled on a fork PR.

`npm test` is not run by any CI job. The three jobs are `sast`, `security-docs` and `build`. The Vitest suite — including the parity tests that hold this codebase's most important invariants — runs only when a human runs it. See backlog item B18.

Migrate before push. `next build` never runs `migrate deploy`, by design. Merging code that `SELECT`s a column production does not have yet breaks the site the moment Vercel finishes deploying. Apply the migration with `npm run db:deploy` against the Supabase URLs **first**, then merge. This is sharper than it looks because of the Gmail rotation tool: with it installed there is no next *manual* deploy to gate on — it will deploy `main` for you within three days, unattended.

Also: pushing anything under `.github/workflows/` needs the `workflow` GitHub token scope (`gh auth refresh -s workflow`); a plain `repo`-scoped token is rejected.

There is no automated pre-push review gate. An `xhigh` review marker used to be enforced by a Claude Code `PreToolUse` hook; both the hook script and its settings entry were removed on 2026-07-30. Running a review before opening a PR is a convention now, not enforcement. A stale `.git/xhigh-review-ok` file may sit in a local clone; it is inert.

3.10 Hosting topology

- **Vercel** runs the Next.js app: Server Components, serverless Route Handlers, and the Node-runtime proxy. Git-integration deploys build on push to `main`.
- **Supabase** provides Postgres. The app uses the **transaction pooler** (port 6543, `pgbouncer=true`) at runtime as `DATABASE_URL`; migrations use the **session/direct** connection (port 5432) as `DIRECT_URL`. Mixing them up is the most common failure.
- **Supabase is used only as a Postgres database.** No Supabase Auth, no Supabase Storage, no Supabase JS client, no anon key. `docs/ARCHITECTURE.md:273–293` explains why Auth.js was chosen over Supabase Auth and why switching would be a rewrite with no benefit for this app.
- **Vercel Hobby quirk:** git deployments are blocked unless the commit's author email is linked to a GitHub account on the Vercel team.

3.11 Landmines and invariants

These are the non-obvious rules a newcomer will break. Each has already cost someone real time. They are drawn from `CLAUDE.md`; the consequence is stated because that is what makes a rule stick.

#	Rule	Consequence of getting it wrong
1	**Never put a layout class on a `<dialog>`.** Put `.card` / `.stack` on an inner wrapper.	Browsers hide a closed dialog with the UA rule `dialog:not([open]) { display: none }`, and **any author rule that sets `display` beats it**. `<dialog className="card stack">` makes every *closed* dialog render. This shipped: `/items` had **50 closed dialogs**, one per row, each a 375×375 absolutely-positioned box, and their `<p>` elements ate the clicks meant for the Delete buttons. `src/components/DeleteItemButton.tsx` is the reference; `ItemSelectTable.test.tsx` pins it. **Do not "fix" this class of bug with a global `dialog:not([open])` rule.**
2	**Do not delete `@source not inline("container")`** from `src/app/styles.css:89`.	Tailwind harvests the candidate `container` from an attribute value *and from English prose in a comment* in `components/ui/table.tsx`, emits a `.container` utility that outranks the `legacy` layer, and widens **every** pre-existing page (720px → 1280px at 1440px, measured). Renaming the slot does not fix it.
3	**Never query inside a loop or `.map`.** No `Promise.all(ids.map(id => prisma…))`.	This is the banned N+1. Batch with `findMany({ where: { id: { in: ids } } })`, fetch relations with `include` / `select`, aggregate with one `groupBy`. `/receipts/new` currently violates it — see B7.
4	**Bound every list.** Every query backing a list must paginate.	`Item` is 1,200+ rows and growing. Never ship a whole table to a Client Component.
5	**The two item-edit surfaces share ONE field definition.**	The item card and the admin edit page both build their admin schema from `editableItemFields` in `items.schema.ts` — exactly seven fields. Add a field there, not to one schema, or the two surfaces drift. `z.object()` **strips unknown keys**, so a field a form renders but the schema does not declare **saves nothing while reporting "Saved"** — that bug has shipped twice.

#	Rule	Consequence of getting it wrong
6	`make` / `model` / `serialNumber` **live in their own schema, action and form** (`itemIdentitySchema` → `updateItemIdentityAction`).	They are deliberately *not* in `editableItemFields`, so they can never reach a non-admin or the item detail card. Keep them separate.
7	**A blank service deadline means NO deadline on CREATE and NO CHANGE on UPDATE.**	Two different questions, two functions. `computeServiceDueAt` returns `null` for blank; `serviceDueAtUpdate` returns `{}` so `dueAt` never appears in the UPDATE at all. That is what makes a no-op re-save exactly stable. Prefilling the days input instead *drifts*: a 7-day deadline set 3 days ago prefills as 4, and saving unchanged pushes it out again, every time. Clearing is reachable only through `setServiceDeadline`.
8	**A NEW ROUND of service resets the deadline; an edit to a LIVE request does not.**	`completeServiceItem` leaves the finished round's `dueAt` / `overdueAlertedAt` on the row, so both paths out of `COMPLETED` must clear them first. Otherwise a device that broke a second time opens as "Overdue 17d" *and* inherits an alert stamp, which the sweep's `overdueAlertedAt: null` filter turns into **this lapse can never alert**.
9	**Rate limiting: spend the token BEFORE the work, refund on success.**	Inverting it to "check first, charge on failure" is a TOCTOU hole exactly as wide as the bcrypt compare — N concurrent POSTs all read an untouched bucket. The refund is what keeps it a *failure* budget: the desk shares one NAT egress IP, so charging successful sign-ins would take everyone offline after five logins.
10	**One capability = one scope.** Spend the NARROW bucket first, the shared ceiling second.	Sharing `login`'s scope meant 60 unauthenticated `POST /api/auth/signout` calls could lock the whole desk out of sign-in. Charging the ceiling first lets 60 requests naming one address lock out everyone behind that egress.
11	`resetRateLimit` **EMPTIES a bucket; it is not a decrement.**	Only on success, only on a bucket carrying the caller's own identity — never a bare `(scope, ip)` key, or one person's success hands everyone on that network a fresh budget. Never on a refusal path.
12	**Never move an interactive form's rate limit into** `src/proxy.ts` **.**	A 429 to a Server Action POST cannot be rendered by `useActionState` and escalates to the error boundary. Return `{ error }` from the action.
13	**The 300/min anti-scraping limit is ANONYMOUS-ONLY.**	Next prefetches the `/i/<id>` links on `/items` and the proxy runs on prefetches, so limiting signed-in staff would 429 the desk out of its own app.
14	`passwordChangedAt` **is the app's only session-revocation lever.**	Any new password-mutation path must stamp it, on the **success** path only. Stamping on a refusal would let anyone who can reach the form log the real owner out. `src/auth.ts:106-137` does the comparison; two tests in `users.service.test.ts` pin both halves.
15	**Never infer sign-in success from a thrown** `NEXT_REDIRECT` **without first refusing** `X-Auth-Return-Redirect` **.**	`signIn()` forwards incoming headers to `@auth/core`, which turns that header into "return the error instead of throwing it" — so a wrong password arrives as a redirect and reads as success.
16	`POST /api/auth/callback/*` **is closed with a 404 and must stay closed.**	It is a second front door to the credential check that bypasses Turnstile, the per-account bucket, and the botnet counter — all of which live in `loginAction`. `signIn()` runs in process; the app never uses it.
17	`src/lib/rate-limit.ts` **must never import Prisma, bcrypt, or** `server-only` **.**	`src/proxy.ts` imports it, and the proxy bundle must stay free of a DB client. Same rule for `public-access-cookie.ts`.

#	Rule	Consequence of getting it wrong
18	**`Item.currentUserEmail` is NOT an email-validated field.**	The CSV importer copies `assignedUser` into it verbatim, so live rows hold values like `"SGT Smith"`. The inputs use `inputMode="email"`, **never** `type="email"` — a browser-side constraint there would block saving the *other* fields on exactly the badly-imported rows the edit form exists to clean up.
19	**`DEPLOYED` and `IN_REPAIR` are absent from the readiness target enum on purpose.**	They are derived, so a POST asking for them is **rejected**, not ignored. Widening that enum is a security change; `admin/actions/readiness.ts` is on the `check-security-docs` watch list for exactly that reason.
20	**Category and unit normalization runs at FIVE write sites** — CSV import, admin edit page, item card, bulk selection bar, new-item form.	All five must call `normalizeCategoryName` *and* `learnCategories`. Miss either half and an item holds a string matching no vocabulary row, so the in-use count under-reports and an admin can delete a category still in use.
21	**`categoryOptional` is IMPORT-only; edit forms use `categoryClearable`.**	The former maps blank → `undefined`, which `diffItemFields` reads as "not submitted" — correct for a partial CSV, wrong for a form, where it made clearing the box a silent no-op that still reported "Saved".
22	**RLS is not the authorization boundary.**	The app reaches Postgres only through Prisma on a privileged role that **bypasses RLS**. Never assume the DB scopes rows for you. Never disable RLS on a table, never add a permissive policy, never `GRANT EXECUTE` on a `public` function to `anon` / `authenticated`.
23	**Public receipts and item pages are enumerable BY DESIGN.**	Do not gate `/receipts/*`, `/receipts/*/pdf`, `/i/*` or the public search behind auth, and do not make receipt identifiers unguessable, when re-auditing. It is an accepted team requirement, boxed in `CLAUDE.md`. Hardening it is a deliberate feature change requiring an explicit request, not a security bug to auto-remediate.
24	**Do not extend the receipt-link token to `/i/*` or any broader grant** without an explicit decision.	`receipt-link-token.ts` is on the `check-security-docs` watch list for exactly that reason.
25	**Docs are part of the change, not a follow-up.**	Any commit that alters behavior, UI, data, an endpoint, a command, an env var, or an architectural rule MUST update the affected documentation **in the same commit**. Any `feat:` / `fix:` MUST add a `CHANGELOG.md` entry under today's date. Any change to authn/authz, crypto/tokens/cookies/secrets, the public surface, retention windows, or CI security posture MUST update `docs/SECURITY.md` and bump its *Last reviewed* date — and that one is enforced by CI. **When you add a new security-relevant file, add it to the watch list at the top of `scripts/check-security-docs.mjs`**, or it silently escapes the guardrail.
26	**Only one agent or developer runs `npm test` at a time.**	It truncates a shared `handreceipt_test` database. Two concurrent runs corrupt each other and masquerade as flaky tests in unrelated files.
27	**Turnstile refuses automated browsers, including Playwright.**	`playwright.config.ts` pins Cloudflare's always-pass test keys. With real keys the e2e sign-in hangs at "Checking your browser…" and looks like a broken login. Never respond to that by weakening the challenge.

4. Future expansion roadmap

Twenty-one tickets, organised by theme. Each states the need, an approach, the files it touches (verified against the code), effort, priority, and its risks and dependencies.

Effort: S ≈ under a day · M ≈ a few days · L ≈ a week or more · XL ≈ an epic.
Priority: P1 = do next · P2 = do soon · P3 = worth doing.

Index

ID	Title	Area	Effort	Priority
B1	Prevent double-checkout of the same item	Functional	**M**	**P1**
B2	Replace the DA 2062's black guard bars with diagonal hatching	UI/UX	**S**	P2
B3	Real progress feedback for the CSV import	UI/UX	S / M / L	P2
B4	Search by name — person and device	Functional	**M**	**P1**
B5	Run the app on a least-privilege database role	Security	**M**	P2
B6	Configure the connection pool explicitly	Data & performance	**S**	**P1**
B7	Dedupe and batch the `/receipts/new` query fan-out (F4)	Data & performance	**S**	P2
B8	Filter by compliance (audit) status and by readiness	Functional	**M**	**P1**
B9	Make the compliance donut actionable; add Audit to the unit leaderboard	UI/UX	**M**	P2
B10	Sub-hand-receipts (**epic**)	Functional	XL	P2
B11	F2 — a non-Latin-1 name permanently breaks that receipt's PDF	Security / correctness	S then M	**P1**
B12	Get the RLS posture into version control, then verify it in production	Security	**M**	**P1**
B13	F3 — validate the receipt signature PNG at write time	Security	**S**	P2
B14	F5 — equalise login timing	Security	**S**	P3
B15	U4 — add baseline security response headers	Security	**S**	P2
B16	An authentication and privileged-mutation event log	Security	**M**	P2
B17	U9 — close the `check-security-docs` guardrail's blind spots	Technical debt	**S**	**P1**
B18	Run the test suite in CI	Technical debt	**S**	**P1**
B19	Broaden end-to-end browser coverage	Technical debt	**M**	P2
B20	U6 — add the target-database guard to `seed-e2e.ts`	Security	**S**	P3
B21	UI audit — accessibility, mobile and consistency	UI/UX	**M**	P2
B22	Resolve the stranded `@testing-library/jest-dom` dependency	Technical debt	**S**	P3
B23	Run a comprehensive dynamic security test with Strix	Security	**M**	P2

Security

B5 — Run the app on a least-privilege database role (defence in depth, not a live IDOR)

> **Read this framing carefully.** The 2026-08-05 assessment found **no reachable IDOR and no authorization bypass**: every one of the 49 Server Actions and 6 Route Handlers gates itself as its first awaited statement, role and activation are re-read from the database on every request, and no action derives identity, role or signature material from client input (`SECURITY_ASSESSMENT.md:224–241`). **Do not describe this ticket as fixing a live vulnerability.** It is defence in depth against a bug that does not exist yet.

Problem. The app connects to Postgres through Prisma on a **privileged role that bypasses RLS** (`SECURITY_ASSESSMENT.md` accepted risk **A5**; `src/lib/prisma.ts:7`; `CLAUDE.md` §6). Consequence, stated plainly: *if an app-layer authz bug is ever introduced, the database will not catch it.* Nothing scopes rows for you, and nothing constrains what a compromised code path can do — including DDL, reading `PasswordResetToken` hashes, or `TRUNCATE`.

Approach.

1. Create a dedicated `app_runtime` role in Supabase: `NOLOGIN`-derived login role, **not** `postgres`, **without** `BYPASSRLS` and **without** `SUPERUSER`.
2. Grant it exactly `SELECT, INSERT, UPDATE, DELETE` on the application tables in `public`, plus `USAGE, SELECT` on sequences (**required** — `createTransfer` calls `nextval('receipt_number_seq')`, `src/modules/transfers/transfers.service.ts:45`).
3. Grant **no** DDL. Migrations already run on a separate connection (`DIRECT_URL`), so keep the migration role privileged and separate.
4. Point `DATABASE_URL` at the new role. `DIRECT_URL` is unchanged.
5. Add `ALTER DEFAULT PRIVILEGES` so tables created by future migrations inherit the grant automatically — otherwise the next migration silently breaks the app at runtime.

Files/config. `src/lib/prisma.ts:7` (no code change needed — the role is in the URL); `prisma/manual/` (a new tracked DDL script); `DEPLOY.md` §2/§4; `docs/SECURITY.md` §10; `CLAUDE.md` §6.

Effort: M. **Priority:** P2.

Risks and dependencies.

- **Pairs with B12.**
  RLS policies mean nothing while the app runs on a bypassing role, and a non-bypassing role with deny-all RLS and no policies would lock the app out completely. **Sequence: B12 first (get the posture into version control and verify it), then design the policies, then de-escalate.** Doing B5 alone against today's "RLS enabled, no policy" tables would take the site down.
- A missed grant surfaces as a runtime `permission denied` on a code path nobody exercised in staging. Mitigate by running the full Vitest suite against a database using the new role before deploying.
- Extensions (`citext`, `pg_trgm`) and their operators need no extra grant, but confirm.

B12 — Get the RLS posture into version control, then verify it in production

Problem. `docs/SECURITY.md:864–867` states that every table is "RLS enabled with no policy = deny-all" and credits an **`rls_auto_enable` event trigger** for making new tables inherit that automatically, citing `prisma/migrations/20260721170000_public_access_setting/migration.sql`.

Verified for this handover: that migration file is 16 lines and contains only a `CREATE TABLE`, one index, and one foreign key — no trigger. A search for `CREATE EVENT TRIGGER`, `ENABLE ROW LEVEL SECURITY` and `CREATE POLICY` across **all** of `prisma/` (41 migrations plus `prisma/manual/`) returns **nothing**. The identifier `rls_auto_enable` appears in exactly four places, all of them comments or a `REVOKE` against it: `prisma/manual/2026-07-20_lockdown_anon_grants.sql:8` and `:20`, `prisma/schema.prisma:523`, and `prisma/migrations/20260721170000_public_access_setting/migration.sql:3`.

The trigger is asserted in three places and defined in none. This is `SECURITY_ASSESSMENT.md` **U3**, and it means **any environment rebuilt from `prisma migrate deploy` cannot reproduce the documented posture.**

There is a second, related gap. `prisma/manual/2026-07-20_lockdown_anon_grants.sql` is a one-shot `REVOKE ALL … FROM anon, authenticated` with **no `ALTER DEFAULT PRIVILEGES`**, so two tables created *after* it — `PublicAccessSetting` (2026-07-21, which holds the PIN's bcrypt hash) and `DeviceCategory` — were never covered and would have regained Supabase's default anon grants.

Approach — in this order.

1. **Capture reality first.** Connect to production and dump the actual state: does `public.rls_auto_enable()` exist? Is there an event trigger on it? Which tables have `relrowsecurity`? What grants do `anon` / `authenticated` hold on `PublicAccessSetting` and `DeviceCategory`?
2. **Write what you find into a tracked migration** — the function, the event trigger, the `ALTER TABLE … ENABLE ROW LEVEL SECURITY` statements, the revokes, and an `ALTER DEFAULT PRIVILEGES … REVOKE ALL … FROM anon, authenticated`. Make it idempotent (`IF NOT EXISTS` / `DROP … IF EXISTS`) so it is safe to apply to the already-configured production database.
3. **Correct** `docs/SECURITY.md:866–867` to cite the real file.
4. **Only then** consider actual `CREATE POLICY` work — and only in the context of B5, because policies are inert against a `BYPASSRLS` role.

Files. New migration under `prisma/migrations/`; `prisma/manual/2026-07-20_lockdown_anon_grants.sql` (supersede or annotate); `docs/SECURITY.md:858–874`; `prisma/schema.prisma:523`; and add `^prisma/` to the watch list in `scripts/check-security-docs.mjs:35–142` (see B17).

Effort: M (roughly 2 hours of DDL plus a production verification session).
Priority: P1 — it is cheap, and it is the item most likely to keep the rest of the security documentation true.

Risks and dependencies.

- **Requires production database access**, which this handover did not have. Step 1 is not optional: writing the migration from the *documentation* rather than from the *database* would encode the same unverified assumption in a new place.
- Applying `ENABLE ROW LEVEL SECURITY` to a table that already has it is a no-op; applying it where the app runs on a bypassing role is also a no-op for the app. Both are safe. Adding a **policy** is not — see B5.

B11 — F2: a non-Latin-1 character in a party name permanently breaks that receipt's PDF

Problem. `partySchema` validates party names with `.trim().min(1)` only — no charset restriction. `partyBlock` then emits the raw name into `page.drawText` using `StandardFonts.Helvetica`, and pdf-lib routes standard-font text through WinAnsi, which **throws** on any code point outside cp1252. The PDF route (`src/app/receipts/[receiptNumber]/pdf/route.ts:6-18`) has no try/catch and Route Handlers have no error boundary, so it returns **500, permanently** — receipts are immutable and no application path rewrites `senderName` / `receiverName`.

No attacker is required. `Kalei‘okalani` (U+02BB ‘okina) and `Nguyễn` (U+1EC5) trigger it, **and this is a Hawaii ARNG property book**. Worse, the failure is silent at creation time: `createReceiptAction` renders the PDF for the notification email inside its own try/catch and only `console.error`s, so the receipt is created and the email goes out without the attachment, and nothing surfaces to the operator.

Approach — two independent changes, both needed.

- **(a) Containment, five minutes.** Wrap `renderReceiptPdf` in `src/app/receipts/[receiptNumber]/pdf/route.ts` so any render failure returns a handled error plus a server log rather than an unhandled 500. Do this **first**.
- **(b) Make the text renderable.** Register `@pdf-lib/fontkit` and embed a Unicode TrueType font with `subset: true` for every `drawText` / `widthOfTextAtSize` carrying user data; or sanitize through a helper using `Encodings.WinAnsi.canEncodeUnicodeCodePoint` before drawing.

Files. `src/modules/receipts/hand-receipt.ts:279-281` (and `:263`, `:265`, `:304`; fallback at `:169`); `src/app/receipts/[receiptNumber]/pdf/route.ts:6-18`; `src/modules/transfers/transfers.schema.ts:21-47`. **The same class affects `src/modules/items/qr-sheet.ts:32-39` via `serialNumber`**, which would break `/admin/items/qr-sheet/pdf` for an entire admin selection — fix both in one pass.

Effort: S for (a), M (about half a day) for (b). **Priority: P1** — this destroys availability of the system's authoritative artifact, permanently, for ordinary names.

Risk. Embedding a font grows every PDF; use `subset: true`. The AcroForm `set()` helper at `hand-receipt.ts:69-96` *is* already wrapped in try/catch, so the finding rests specifically on the unwrapped custody-page draws — do not assume form-field population is affected.

B13 — F3: validate the receipt signature PNG at write time

Problem. `receiverSignature` is validated with **prefix + length only** (`src/modules/transfers/transfers.schema.ts:67-70`), capped at `MAX_SIGNATURE_BYTES = 5_000_000`. The app has a shared signature validator, `signatureError` (`src/lib/signature.ts:5`), capping at `MAX_SIGNATURE_LEN = 250_000` — used on three other paths (account saved signature, returns action, saved-signature schema). **The receipt path alone skips it, at 20× the size, on the one signature path whose output is publicly reachable.**

A ~100-byte PNG declaring 12000×12000 RGBA makes `UPNG.decode` allocate ~576 MB straight from the attacker's IHDR, with no dimension sanity check. The `try/catch` around the decode catches the JS throw, so this is not a permanent 500 — but `_filterZero`'s unconditional `O(width × height)` loop is not catchable, so every subsequent `GET /receipts/<n>/pdf` costs multi-second CPU and hundreds of MB RSS from a stored ~100-byte value. And the 300/min budget is anonymous-only by design, so an authenticated account can hammer it unmetered.

Approach. Validate at **write** time, not render time. Route `receiverSignature` through the shared `signatureError`, and extend `src/lib/signature.ts` to base64-decode the payload, check PNG magic bytes, and read IHDR width/height — rejecting anything whose pixel count exceeds a signature-sized bound. Putting it in `signatureError` means **all four** entry points inherit it and the invariant that `src/modules/signatures/signatures.schema.ts:4-6` already *claims* becomes true.

Files. `src/lib/signature.ts`; `src/modules/transfers/transfers.schema.ts:5, :67-70`; `src/modules/signatures/signatures.schema.ts:4-6` (correct the comment).

Effort: S (about an hour). **Priority:** P2.

Risk. Lowering the cap could reject a legitimately large signature from a high-DPI tablet. Measure a real `SignaturePad` output before picking the bound.

B14 — F5: equalise login timing to close the account-enumeration oracle

Problem. `src/auth.ts:35-37` returns after one indexed `findUnique` for an unknown or deactivated account, but proceeds to bcrypt (cost 12, ~200–400 ms) for a live one. Both branches return byte-identical text and both call `recordAuthFailure`, so **only the bcrypt term differs**. Turnstile runs before `signIn()` and is therefore common-mode latency on both branches. An attacker classifies a list of addresses as staff/non-staff, converting a blind spray into a targeted one; because the email differs per probe the narrow `(ip, email-hash)` bucket is fresh each time and only the 60/IP/15min ceiling binds.

Approach. When no active user is found, compare the submitted password against a fixed dummy bcrypt hash at the same cost before returning `null`. This mirrors treatment the reset surface already received deliberately — see `src/app/actions/auth.ts:307-343`, commented

"FIX #2 (timing side-channel)".

Files. `src/auth.ts:35-37`. **Effort:** S. **Priority:** P3 — real but low yield against a small admin-provisioned roster that is throttled and CAPTCHA-gated.

Dependency. `src/auth.ts` is on the `check-security-docs` watch list, so this needs a `docs/SECURITY.md` update in the same commit.

B15 — U4: add baseline security response headers

Problem. `next.config.ts:5-38` sets only `Referrer-Policy: no-referrer`, on two path groups (the reset flows, and `/receipts/*` for the `?k=` capability token). There is **no CSP, no `frame-ancestors`, no `X-Frame-Options`, no `X-Content-Type-Options`, and no HSTS** anywhere.

The assessment cleared this unanimously as *not a live hole*: Auth.js defaults the session cookie to `SameSite=Lax` and the app never overrides it, so a cross-origin iframe carries no session and renders logged-out — clickjacking of admin controls fails. And there is no `dangerouslySetInnerHTML` anywhere in the tree, so a missing CSP has no demonstrated sink.

Approach. Add the four headers as defence in depth. Start CSP in `Report-Only` — Next injects inline scripts and styles, so a strict policy needs nonce plumbing and will otherwise break the app in ways `next build` cannot catch.

Files. `next.config.ts:5-38` (**on the watch list** — needs a `docs/SECURITY.md` update in the same commit).

Effort: S for the three simple headers, M if you pursue a real CSP.
Priority: P2.

B16 — An authentication and privileged-mutation event log

Problem. `docs/SECURITY.md` names this the **biggest gap** in its own at-a-glance table (`:25`), and Known gap 7 records it. There is no log of sign-ins, sign-in failures, lockouts, password resets, or PIN unlocks; no IP or user-agent capture anywhere; and most privileged mutations record no actor — *who retired this device*, *who closed that ticket*, *who promoted this account* are all unanswerable.

The assessment additionally notes the register's own list is **incomplete** (U8): it omits `deleteItemAction` (`src/app/admin/actions/items.ts:267-285`), the most destructive action in the app — permanent, no undo, no server-side refusal even for an item on an open receipt (accepted risk A14).

Approach.

1. Add an `AuditEvent` table: `actorId?`, `actorName` (denormalized, like `ItemEdit`), `action`, `subjectType`, `subjectId`, `metadata Json`, `ip?`, `userAgent?`, `createdAt`. Nullable actor with `ON DELETE SET NULL` plus a name snapshot, following the existing `ItemEdit` / `ReturnTransaction` pattern so history survives account deletion.
2. Write from the existing action layer, in the same transaction as the change.

3. Cover authentication separately (`loginAction`, the reset flow, `/unlock`) — note these run on paths where a DB write per failed attempt is itself a DoS lever, so consider sampling or writing only state transitions.
4. Add a retention window and fold it into the existing purge worker, or this table grows without bound.

Files. `prisma/schema.prisma`; `src/app/admin/actions/*.ts` (items, users, queue, units, categories, readiness, receipt-timer); `src/app/actions/auth.ts`; `src/app/actions/unlock.ts`; `src/app/api/cron/purge/route.ts`; `docs/SECURITY.md` Known gap 7 and A13.

Effort: M. **Priority:** P2 — high value, but it is new surface rather than a fix, and it should not jump ahead of B11/B12.

Risk. An event log holding IPs and emails is itself PII with a retention obligation. Decide the window *before* building it, not after.

B20 — U6: add the target-database guard to `seed-e2e.ts`

Problem. `prisma/seed-e2e.ts:10-12` creates an `ADMIN` with a hardcoded password and has **no target-database host allowlist**. Its sibling `prisma/seed-analytics-demo.ts:33-56` implements exactly that guard, with a header comment explaining why guarding on `NODE_ENV` is not enough (`tsx` leaves it unset). **The project set its own standard and did not apply it to the more dangerous script.**

Nothing invokes it automatically — not CI, not Playwright — so it requires operator error. But the operator error in question is "ran `npm run db:seed:e2e` with production URLs in the shell", which is exactly the mistake the sibling guard exists to prevent.

Approach. Copy the resolved-`DATABASE_URL` host allowlist from `seed-analytics-demo.ts:33-56` into `seed-e2e.ts`.

Files. `prisma/seed-e2e.ts:10-12`. **Effort:** S (about ten minutes). **Priority:** P3.

Related — the docs half is now fixed, the production half is not. The security assessment recorded that `README.md` and `.env.example` still advertised `admin@example.com / ChangeMe123!` as seed defaults when `prisma/seed.ts:24-29` had already been changed to **throw** without `SEED_ADMIN_*`. Both files were corrected in the 2026-08-05 documentation sync and now state the vars are required with no default account.

What that does **not** settle: git history shows the default was live between `cfe582d` (2026-06-30) and `5a82658` (2026-07-06), with the production deploy `ea27153` falling *between* those commits. **Action item, still open: check production for a legacy `admin@example.com` account and remove it if present.** This handover has no database access and could not check.

B23 — Run a comprehensive dynamic security test with Strix

Problem. The security work to date is a **static** review plus a **targeted** dynamic pass that exercised only the three findings F2/F3/F5 by hand (see `SECURITY_ASSESSMENT.md`). Neither is a full dynamic application security test: no tool has crawled the running app end-to-end,

fuzzed its inputs, or systematically attempted auth/authz bypass, injection, and resource-exhaustion across every route. Breadth is the gap — a static review only finds what a reader thinks to look for, and the targeted pass only confirmed what was already suspected.

Approach. Run **Strix** — a dynamic application security testing / autonomous penetration-testing tool — against a **local instance, never production**, for a far more comprehensive dynamic pass than this repo has had. It should exercise the running app across the classes this project cares about: the public surface behind the PIN gate, the per-receipt capability token, the rate limiters, the CSV/import endpoints, and the PDF/QR routes (F2/F3 live there). Treat its output as candidate findings to triage the same way the static assessment's were — reachability, impact, then an adversarial "is this real" check — rather than filing raw tool output.

Guardrails, non-negotiable.

- **Local only.** Never point it at `www.dcsim.us`. Dynamic testing that includes brute-force and fuzzing against a live system serving HIARNG is disruptive by design and is not authorised without an explicit decision.
- The dev server permits one instance at a time and the test DB truncates under concurrent runs (see §3.2), so it cannot share the machine with active development.
- Get authorization in writing before any run that touches a shared or deployed environment.

Files/scope. No source change — this is an activity, not a code edit. Its findings become their own tickets. **Effort:** M (a run plus triage). **Priority:** P2 — the static and targeted-dynamic passes are done and clean, so this is the natural next depth, not an urgent gap. Pairs with **B19** (a driven browser) and complements, rather than replaces, Semgrep and dependency scanning.

Functional features

B1 — Prevent double-checkout of the same item

Problem / user need. An item can be issued on two open hand receipts at the same time. Two people can each hold a signed DA 2062 for the same laptop, and the property book cannot say which is authoritative.

State this precisely, because it was investigated for this handover: no guard exists anywhere — not in the application and not in the database.

- `createTransfer` (`src/modules/transfers/transfers.service.ts:23-106`) validates exactly three things, at `:26-28`: that every id resolves to an item, that the count matches, and that none is `RETIRED`. **It never queries custody.** There is no `SELECT … FOR UPDATE`, no advisory lock, and the `TransferError` code union (`src/modules/transfers/transfers.errors.ts:2`) is closed and contains **no** "already issued" code — so there is not even an error to throw.
- **No database constraint.** `TransferItem` (`prisma/schema.prisma:331-349`) carries `@@index([itemId])` and `@@index([transferLineId])` — plain indexes, no `@unique`. Verified against all 41 migrations plus `prisma/manual/`: the only unique index on any `itemId` in the tree is `ServiceQueueItem_itemId_key`.

- The only dedupe is **within a single receipt** (`transfers.service.ts:31–34`), and its own comment concedes the gap: *"nothing in the schema blocks that at the DB level."*
- **The picker does not filter.** `/receipts/new` filters on lifecycle only (`src/app/receipts/new/page.tsx:16` : `.filter(i => i && i.status === "ACTIVE")`). The row checkbox in `ItemSelectTable.tsx:133` is rendered on `status === "ACTIVE"` alone; the Holder column at `:138` is display-only. The item page's "Create hand receipt" link (`src/app/i/[itemId]/page.tsx:99`) is gated on `loggedIn && status === "ACTIVE"` and **not** on `currentHolder` , which that same page has already fetched at `:41` .
- **The only custody-aware affordance is a client-side cosmetic warning**, `src/app/receipts/new/ReceiptBuilderForm.tsx:449–457` — "Held by X, not Y". Its own comment says *"Added, never blocked."* It is suppressed when the sender name is blank or happens to match, and it has no server-side counterpart, so a crafted POST bypasses it entirely, as does simply ignoring it.
- **Downstream code already tolerates the state rather than preventing it.** `src/modules/transfers/holders.query.ts:23` — *"DISTINCT ON picks the most recent open receipt **if an item somehow sits on two***". `src/modules/items/readiness.sql.ts:31–33` — *"the open-receipt test is an EXISTS … **so an item on two receipts still counts once***".

  The consequence of that tolerance is the real bug: on a double-issue, the item page and the Holder column show only the *newest* open receipt's receiver. The older open receipt silently disappears from every "who holds this" surface while remaining `OPEN` and returnable.

Approach. Three layers, in this order.

1. **A friendly pre-check.** Inside `createTransfer` 's existing transaction, before the create, query open custody for `uniqueIds` using the shared predicate (`src/modules/transfers/custody.sql.ts:44` — `ti."returnedAt" IS NULL AND t."status" = 'OPEN'` , with `CUSTODY_FROM` at `:35–38`). Add an `ALREADY_ISSUED` code to `transfers.errors.ts:2` and map it to user copy in `src/app/actions/receipts.ts` (the existing `TOO_MANY_LINES` mapping at `:114` is the pattern). Name the offending serials and receipt numbers in the message.
2. **A race-safe backstop.** A read inside a READ COMMITTED transaction does not stop a concurrent insert, so the pre-check alone is TOCTOU. Add a **partial unique index**: `CREATE UNIQUE INDEX … ON "TransferItem" ("itemId") WHERE "returnedAt" IS NULL AND "itemId" IS NOT NULL;` and catch Prisma `P2002` into the same friendly error — the same pattern the CSV importer already uses for serial collisions.
3. **Stop offering it in the UI.** Disable (do not hide) the checkbox in `ItemSelectTable.tsx:133` for an item with a live holder, with an explanatory title; make the `ReceiptBuilderForm.tsx:449–457` warning unconditional on holder presence rather than conditional on a sender-name mismatch.

Files. `src/modules/transfers/transfers.service.ts:23–106` ; `src/modules/transfers/transfers.errors.ts:2` ; `src/modules/transfers/custody.sql.ts` ; `src/app/actions/receipts.ts:82, :114–115` ; `src/app/receipts/new/page.tsx:16` ;

`src/components/ItemSelectTable.tsx:133`; `src/app/receipts/new/ReceiptBuilderForm.tsx:449-457`; `src/app/i/[itemId]/page.tsx:99`; a new migration.

Effort: M. **Priority: P1** — this is a correctness hole in the system's core promise.

Risks and dependencies.

- ⚠️ **Hard dependency with B10.** A sub-hand-receipt is *legitimately* the same item on two open receipts — the parent (accountable) and the child (physical). **A naive partial unique index forbids sub-receipts outright.** Either decide B10's data model first, or scope the index/pre-check to exclude a parent-child pair from the start. Do not ship step 2 without making that decision consciously.
- ⚠️ **The index may fail to create against existing data**, for two reasons: real duplicates may already exist, and any historical row on a `CLOSED` receipt with `returnedAt IS NULL` would participate. Run the detection query first, reconcile, and only then add the index.
- Step 3 changes what a technician can select. Disable rather than hide, and say why — silently missing rows read as a broken list.

B4 — Search by name — person and device

Problem / user need. "Find everything Jane Doe has" and "find the laptop by its device name" must work from the surfaces people actually use.

What already exists (verified).

Surface	Matches	Where
Home search, item mode	`serialNumber` **only**	`searchItemsBySerial`, `src/modules/items/items.service.ts:775` — one `$queryRaw`: `WHERE "serialNumber"::text ILIKE …`, `LIMIT 50`
Home search, receipt mode	`receiptNumber` **only**	`searchReceiptsByNumber`, `src/modules/transfers/transfers.service.ts:117` — `contains`, insensitive, `take: 50`, selecting only `receiptNumber` + `itemSummary`
`/items ?q=`	`deviceName`, `make`, `model`, `serialNumber`, **plus the recipient named on the item's current open hand receipt**	Prisma path `items.service.ts:323-357`; raw path `itemFilterSql`, `items.service.ts:231-241`; `recipientMatchSql` `:198-208`; tokenizer `src/modules/items/recipient-search.ts:52-59`
Receipt builder autofill	`firstName`, `lastName`, `email`, `unit` on `Contact`	`searchContactsAction` → `searchContacts`, `src/modules/contacts/contacts.service.ts:27-46`

So **device-name and holder-name search already work on `/items`** — for signed-in users, tokenized so "doe jane" finds "Jane Doe", with edge punctuation stripped so "Doe, Jane" works.

What is genuinely missing.

1. **The public home search covers neither.** A recipient who knows their own name but not the serial has no way in. `liveSearchAction` (`src/app/actions/search.ts:32-38`) offers exactly two modes, serial and receipt number.
2. **`?q=` does not match `Item.currentUserEmail`.** That is the MDM "assigned user" field, which per accepted risk A7 holds values like `"SGT Smith"`. So a device assigned only by

import, with no hand receipt, is **not** findable by the person's name — `CHANGELOG.md` states this explicitly as a known limitation of the 2026-07-31 recipient search.

3. **Only *live* custody matches.** The recipient branch is scoped to `OPEN_CUSTODY_PREDICATE`, so "what did Jane Doe have last year" is unanswerable from `/items`.
4. **Devices are not searchable by home unit** free-text (only by the `?uic=` dropdown).

Approach — three separately shippable slices.

- **Slice A (S).** Add `Item.currentUserEmail` to the `?q=` OR-branch on **both** paths. Note the column is `@map("currentUser")`, so the raw side must name the **physical** column `i."currentUser"`. Add a trigram GIN index for it (there is none today). Extend the `FILTERS` array in `items.readiness-sort.parity.test.ts:83-97`.
- **Slice B (M).** Add a device-name / holder-name mode to the public home search. **This is a security decision, not just a feature** — it widens the accepted-public surface (A1) to make people's names and device names enumerable by anyone past the shared PIN. Requires an explicit team decision, a `docs/SECURITY.md` update, and probably a narrower projection than `/items` returns.
- **Slice C (M).** Optional historical mode — match recipients on *closed* receipts too, behind an explicit toggle, since it changes what "found" means. Bear in mind closed receipts are purged after 90 days, so the history is bounded regardless.

Effort: M overall. **Priority: P1** for slice A, P2 for B and C.

Risks.

- ⚠️ **Both query paths or neither.** `items.readiness-sort.parity.test.ts` asserts the Prisma and raw paths return identical ids and totals for the same filters. Change one and it fails — which is the guardrail working, but plan for it.
- `searchItemsBySerial` escapes LIKE metacharacters (`items.service.ts:783`) while `itemFilterSql` deliberately does **not**. Do not "harmonise" that without understanding both choices.
- Every new OR-branch needs an index or the `/items` list gets slower for everyone. Trigram GIN for `ILIKE '%q%'`; a B-tree will not help.

B8 — Filter by compliance (audit) status, and by readiness

Problem / user need. "Show me everything that is overdue for audit" and "show me everything ready to deploy" are the two questions the property book gets asked most, and neither is expressible today.

What exists. `/items` reads **exactly five** querystring params — `q`, `sort`, `dir`, `page`, `uic` (`src/app/items/page.tsx:18-24`). The only true *filter* control is the Unit (UIC) `<select>` (`src/components/ItemSelectTable.tsx:267-277`). **Readiness and audit state are sort-only** — display columns (`ItemSelectTable.tsx:146`, `:148`) with a ranked sort behind them, but no narrowing. Column *visibility* is a client-side localStorage preference, not a URL param, and hiding a column does not stop it being sortable or filterable.

"Compliance" maps cleanly onto the existing derived **audit state**: `auditState` (`src/modules/audit/audit.status.ts:35-40`), the three values in `AUDIT_ORDER` (`:30`), `AUDIT_PERIOD_YEARS = 1` (`:7`), `auditCutoff` (`:57-61`), and the SQL rank `auditRankSql` (`src/modules/audit/audit.sql.ts:58-61`).

Approach — audit first, readiness second, because they are very different in difficulty.

- **`?audit=compliant|overdue|never` (S–M).** Expressible on **both** paths without new joins, because the state is a function of one column and `now`: `compliant` → `lastAuditedAt > cutoff`; `overdue` → `lastAuditedAt <= cutoff`; `never` → `lastAuditedAt IS NULL`. Prisma side: a `where` clause built from `auditCutoff(now)`. Raw side: extend `itemFilterSql` (`items.service.ts:231-241`), reusing `auditCaseSql` (`audit.sql.ts:32-38`) so the filter and the badge cannot disagree about the boundary.
- **`?readiness=…` (M).** Harder, because readiness spans `ServiceQueueItem` and `Transfer`. The pragmatic implementation is to **route any readiness-filtered query down the existing raw path** (the one already built for the derived *sort*), reusing `READINESS_CASE` / `READINESS_JOINS`. The Prisma path then needs either an equivalent nested-relation `where` or an explicit rule that this filter forces the raw path — decide and document which, because the parity test compares them.
- **UI:** two `<select>` controls beside the existing UIC dropdown in `ItemSelectTable.tsx`, written into the URL by `ItemsSearchInput`'s existing URL rebuild (`src/app/items/ItemsSearchInput.tsx:50-68`, which already carries `q` / `sort` / `dir` / `uic` and resets `page`).

Files. `src/app/items/page.tsx:18-24`; `src/modules/items/items.service.ts:231-241` and `:323-390`; `src/modules/audit/audit.sql.ts`; `src/components/ItemSelectTable.tsx:267-277`; `src/app/items/ItemsSearchInput.tsx:50-68`; `src/modules/items/items.readiness-sort.parity.test.ts:83-97`.

Effort: M. **Priority: P1** for `?audit=`, P2 for `?readiness=`.

Risks.

- ⚠️ **`items.readiness-sort.parity.test.ts` will fail if you change only one path.** That is the point of it. Add the new filters to its `FILTERS` array in the same commit.
- Audit state is **time-dependent**, so the filter must compute the cutoff at request time, from the same `auditCutoff` the badge uses. Hard-coding a date, or computing it twice, reintroduces exactly the drift `auditCutoff` exists to prevent.
- ⚠️ **Do not resolve this by storing readiness or audit state on `Item`.** That is the `deployableStatus` column dropped in `20260728000000_derive_readiness`, and the `isAccountedFor` flag dropped before it. See §3.6.

B10 — Sub-hand-receipts (epic)

Problem / user need. Today custody is flat: the desk issues to a holder, and that holder is the end of the chain. In practice a primary holder — a unit supply sergeant, say — receives equipment and then issues it down to individual soldiers. The desk needs to see that chain

while **the parent receipt remains the accountable record**: the primary holder still owes the desk the equipment, regardless of who is physically holding it.

This is the largest item in the backlog and should be treated as an epic, not a ticket.

The model to decide first. A sub-receipt is a `Transfer` whose sender is the primary holder and whose parent is another `Transfer`. Schema shape:

```
parent   Transfer?  @relation("SubReceipts", fields: [parentTransferId], references: [id], onDelete:
Restrict)
parentTransferId String?
children Transfer[] @relation("SubReceipts")
```

`onDelete` is a real decision, not a default — see the purge sub-task below.

What it disturbs, with the specific code:

Area	What changes
Custody derivation	`getHoldingTransfer` (`transfers.service.ts:148–162`) takes the item's **latest** receipt. With sub-receipts that becomes the *sub*-holder — which is right for "who is physically holding it" and wrong for "who is accountable to us". You need **two** answers where there is currently one. `getLastReceiver` (`:164–175`) prefills the builder's sender, so getting this wrong prints the wrong name on a signed DA 2062.
The shared SQL predicate	`custody.sql.ts:44` (`returnedAt IS NULL AND status = 'OPEN'`) is embedded in three places: `READINESS_CASE` (`readiness.sql.ts:51–56`), `recipientMatchSql` (`items.service.ts:205`), `holdersForItems` (`holders.query.ts:38–40`). Readiness survives unchanged — the open-receipt test is an `EXISTS`, so an item on both a parent and a child still reads `DEPLOYED` once (`readiness.sql.ts:31–33`). But `holdersForItems`'s `DISTINCT ON`-most-recent rule (`holders.query.ts:23`) would show the **sub**-holder in the `/items` Holder column, and the `?q=` recipient search would match either. Decide deliberately, and document.
DA 2062 output	The sub-receipt is its own DA 2062 with the primary holder as sender. The **parent** receipt should show which of its items are sub-issued and to whom. `render.ts:12–24` caps the quantity series at 6 columns and `:52–56` allocates B–F to partial returns only, so there is no spare column — a parent-side indication needs new layout, probably on the custody-record page.
Returns	`processReturn` (`returns.service.ts:23–97`) locks **one** receipt (`SELECT … FOR UPDATE` on `receiptNumber`, `:32–34`) and compare-and-swaps on `TransferItem.id` (`:55–61`). Neither mechanism sees a sibling receipt. Returning an item on the **parent** while a child still holds it open must be refused, or must cascade — a product decision with a transaction-scope consequence (you would need to lock both rows in a deterministic order to avoid deadlock).
Purge / retention	Closing a receipt stamps `purgeAfter = closedAt + 90 days` and `purgeExpiredTransfers` hard-deletes. If a child FKs its parent, deleting a purged parent either cascades (destroying a still-live child) or is restricted (leaving un-purgeable rows forever). Neither default is acceptable; the rule needs to be *"a parent cannot close while a child is open"*, enforced in `assertTransferOpen`'s neighbourhood, plus a purge order.
Double-checkout (B1)	A parent+child pair is *legitimately* one item on two open receipts. Any constraint B1 adds must exempt it.

Sub-tasks.

1. **Product decision document (S).** Accountable vs physical holder — which one the item page shows, which the `/items` Holder column shows, which the receipt builder prefills as sender, what a search for a name should return. Nothing else can start until this is written down.
2. **Schema + migration (S).** The self-relation, an index on `parentTransferId`, and the chosen `onDelete`.

3. **Creation flow (M).** A "sub-issue from this receipt" entry point on `/receipts/<n>` that pre-fills the builder's sender from the parent's receiver and restricts the item picker to that parent's unreturned items.
4. **Custody derivation (M).** Split `getHoldingTransfer` into an accountable-holder and a physical-holder query, update all four call sites (`i/[itemId]/page.tsx:41` , `receipts/new/page.tsx:33` , `actions/scan.ts:32` , `transfers.service.ts:180`), and decide what `holders.query.ts` shows.
5. **DA 2062 rendering (M).** Parent-side sub-issue indication and child-side parent reference.
6. **Returns and close rules (M).** Refuse-or-cascade, plus the lock ordering.
7. **Purge and retention (S).** Close-order rule and purge order.
8. **UI (M).** A custody chain on the item page; parent/child links on the receipt page.
9. **Tests (M).** Extend `readiness.parity.test.ts` and `items.readiness-sort.parity.test.ts` fixtures with parent/child rows, and add integration tests for the return and purge rules.

Effort: XL. **Priority:** P2 — high value, but sequence it after B1's *decision* and after the P1 correctness items.

Risk. This touches the one part of the system that is a legal-ish record. Every change to custody derivation risks printing the wrong name on a signed document. Sub-task 1 is not ceremony; it is the thing that prevents that.

UI / UX

B2 — Replace the DA 2062's black guard bars with diagonal hatching

Problem / user need. The hand receipt prints two solid black bars in the quantity column — above and below the vertical recipient signature — to stop anyone adding rows or a second signature to the empty space. They work, but they consume a lot of toner on every printed receipt, and receipts are printed constantly.

Where it is (verified). Exactly two draws, both `page1.drawRectangle({ …, color: black })`, inside the local closure `drawColumnSig` in `buildHandReceiptPdf`, at `src/modules/receipts/hand-receipt.ts:172-179` :

```
// Guard bars: black out the empty column below and above the signature block.
const gx = cx - 11;
if (sigBottom - 4 - tableBottomY > 1) {
  page1.drawRectangle({ x: gx, y: tableBottomY, width: colWidth, height: sigBottom - 4 - tableBottomY,
color: black });
}
if (lastRowBottom - (blockTop + 2) > 1) {
  page1.drawRectangle({ x: gx, y: blockTop + 2, width: colWidth, height: lastRowBottom - (blockTop + 2),
color: black });
}
```

`:175` is the lower bar (below the signature, down to the table's bottom rule); `:178` is the upper bar (above the signature block, up to the bottom of the last item row). `drawColumnSig` is called once per signed column from the loop at `:189–192`; column A is the issuance signature and B–F are partial returns.

Every geometry variable a hatch generator needs is already computed:

Variable	Line	Meaning
`black`	`:124`	`rgb(0, 0, 0)`
`colWidth`	`:125`	`23` — bar width
`tableBottomY`	`:125`	`58` — bottom edge of the lower bar
`colCenters`	`:130`	`[632, 657, 682, 707, 731, 756]` — the A–F quantity columns
`gx`	`:173`	`cx - 11` — bar left edge
`lastRowBottom`	`:149`	top edge of the upper bar
`sigBottom`	`:150`	top edge of the lower bar (minus 4)
`blockTop`	`:152`, reassigned `:162` / `:170`	bottom edge of the upper bar (plus 2)

So each bar is fully described by `x = cx - 11`, `width = 23`, and a `[yBottom, yTop]` pair.

Approach. Replace each `drawRectangle` with a loop emitting `page1.drawLine(...)` diagonals across the same rectangle — a 45° hatch at a fixed pitch (say 4–6 pt), clipped to the box by computing each line's endpoints against the rectangle bounds. `drawLine` is already available and already used in this file (`:303`, the signature rule on the custody-record page), so no new import is needed. Consider a single thin border rectangle (`borderColor` + `borderWidth`, no `color`) around the hatched area so the guarded region still reads as deliberate.

`qr-sheet.ts` needs no change — it contains **no filled rectangles at all** (only `drawImage` and `drawText`).

Effort: S. **Priority:** P2.

Risks.

- ⚠️ **No test pins the guard bars.** There is no `drawRectangle` assertion anywhere in `src`, so a regression here would ship silently. Add one — asserting the number of `drawLine` calls and their bounding box is enough — and verify by opening a generated PDF.
- The bars are an **anti-tamper** control, not decoration. Hatching must be dense enough that a signature or a row cannot be convincingly drawn over it. Print a sample before deciding the pitch; screen review is not sufficient.
- Both draws are already guarded by a `> 1` height check, so degenerate regions are skipped — preserve that guard.

B3 — Real progress feedback for the CSV import

Problem / user need. A full-fleet import can occupy most of a 60-second function budget, and during that time the operator sees **only a disabled button reading "Importing…"**.

What exists (verified). `src/app/admin/items/import/ImportItemsForm.tsx` is a client component driving a hand-rolled four-state machine — `const [phase, setPhase] = useState<"idle" | "busy" | "resolve" | "done">("idle")` at `:40`. Feedback is button text only: `{phase === "busy" ? "Analyzing…" : "Analyze CSV"}` at `:223`, `{phase === "busy" ? "Importing…" : `Import ${…} items`}` at `:153–155`. There is **no** `useActionState`, **no** `useTransition`, **no** `useFormStatus`, **no** `<Progress>`, and no `aria-live` busy announcement (the only `role="alert"` is the error paragraph).

The transport is **Server Actions invoked directly as async functions** with hand-built `FormData` (`:55–57`, `:72–75`) — not `<form action={…}>` and not `fetch` to a route. That matters: **a Server Action call gives the client no progress events.** Note also the file is uploaded **twice**, once per step, as the component's own comment at `:14–16` records.

A `<Progress>` primitive already exists (`src/components/ui/progress.tsx`, wrapping `@radix-ui/react-progress`), and its docblock at `:8–25` explicitly sanctions a numeric value for "an N-of-M batch" — but it has exactly **one** consumer today, `src/components/HomeSearch.tsx:87–91`, used as an indeterminate sweep for the type-ahead.

Approach — three genuinely different levels of ambition. Pick one deliberately.

- **Level 1 — honest indeterminate indicator (S).** Render the existing `<Progress value={null}>` during `phase === "busy"`, plus an `aria-busy` / `role="status"` announcement. This reports the wait rather than shortening it, exactly as the home search does. It does not lie about completion, and it is a real improvement over a static button.
- **Level 2 — real upload-byte progress (M).** Move the commit step from a Server Action to a Route Handler and post it with `XMLHttpRequest`, whose `upload.onprogress` gives genuine bytes sent. `fetch` does not expose upload progress. This gives a **truthful** bar for the upload phase and must switch to indeterminate once the bytes are sent and the server is working — which, on a local network with a 5 MB cap, is most of the wall clock.
- **Level 3 — true row progress (L, and it costs something).** ⚠️ `commitImport` **runs as a single transaction** (`maxWait` 5s + `timeout` 40s, called out in `DEPLOY.md` §7a), which is what gives the import its "a non-200 means nothing was written" guarantee. Row-level progress requires either chunking the import into multiple transactions — **which destroys that atomicity guarantee and the documented rollback story** — or a side channel that reports progress from inside an uncommitted transaction. Do not do this without an explicit decision to trade atomicity for feedback.

Files. `src/app/admin/items/import/ImportItemsForm.tsx:40, :50–78, :153–155, :223`; `src/components/ui/progress.tsx`; `src/app/admin/actions/items.ts:294` (`analyzeImportAction`), `:311` (`commitImportAction`); for level 2 a new Route Handler modelled on `src/app/api/items/import/route.ts` (note its `maxDuration = 60` at `:42` and the reasoning at `:12–41`).

Effort: S / M / L by level. **Priority:** P2 — level 1 is nearly free and should just be done.

Risk. Level 2 introduces a **second** import front door, and `CLAUDE.md` is emphatic that there are *two front doors and one implementation*. Any new route must call `commitImport`, not fork it, and must gate with `requireAdmin()`.

B9 — Make the compliance donut actionable, and add Audit to the unit leaderboard

> **Check this before writing any code: the pie chart already exists.**

What exists (verified). The analytics dashboard already renders an **audit-readiness donut**: `AuditReadinessWidget` at `src/app/admin/analytics/widgets.tsx:61–125`, using `DonutChart` (`src/app/admin/analytics/charts.tsx:48–101`, a recharts `PieChart` / `Pie`), fed by `getAuditReadiness(scope)` (`src/app/admin/analytics/analytics.service.ts:109–128`) — one `$queryRaw` grouping `ACTIVE` items by `auditCaseSql(cutoff)` and always emitting all three `AUDIT_STATE_ORDER` slices. The centre hole carries a `%` label (`widgets.tsx:106–109`) and an icon+count row sits below (`:112–122`) so identity never rests on hue alone.

So this ticket is not "build a pie chart." What is genuinely missing is three things:

1. **It is not clickable.** The donut has no `onClick`, no `Link`, no router use. The only clickable chart element on the whole dashboard is the unit-allocation table row button (`widgets.tsx:368–381`). Clicking "overdue" should take you to `/items` filtered to overdue — which is precisely what B8 makes expressible.
2. **There is no per-unit audit breakdown.** The donut is fleet-wide within the global `?uic=` / `?unit=` scope — a single `GROUP BY 1` on state, with no unit dimension. The unit leaderboard (`widgets.tsx:342–346`) breaks down **Total / Deployed / Ready** — readiness only, no audit column. Today the only way to get audit-by-unit is to click a unit row to set the scope and re-read the donut.
3. **It is admin-only.** `/admin/analytics` calls `requireAdmin()`. If a `USER` needs to see compliance at a glance, that is an **authorization change**, not a chart change, and needs an explicit decision.

Approach.

- **(a)** Make the slices and the count row navigate to `/items?audit=<state>` (**depends on B8**). Use the existing `setParam` pattern from the leaderboard.
- **(b)** Add an Audit breakdown to the leaderboard: extend the second leaderboard query with `auditCaseSql`, and add columns. Note the leaderboard is deliberately **two** queries (top-N units, then their breakdown) rather than one capped grouping, because capping a two-column grouping slices through the middle of a unit and under-reports its total — preserve that shape.
- **(c)** If a non-admin summary is wanted, put a compact three-number card on `/items` fed by the same `getAuditReadiness`, and make the authorization decision explicitly.

Files. `src/app/admin/analytics/widgets.tsx:61–125, :342–346, :368–381`; `src/app/admin/analytics/analytics.service.ts:109–128`; `src/app/admin/analytics/analytics.types.ts:93, :102, :112–117`; `src/app/admin/analytics/palette.ts:76–80`.

Effort: M. **Priority:** P2. **Dependency:** (a) needs B8.

Risk.

- ⚠️ **The palette is validated, not chosen by eye.** The donut is blue → yellow → red (`#2a78d6` / `#eda100` / `#d03b3b`, `palette.ts:76–80`) **because any green-vs-red pairing is**

indistinguishable under deuteranopia. **Do not propose or "restore" green/red.** Re-run the validator against the ledger surface (`#fbfcf9`) before changing any hex. Note the `/items` audit dots deliberately use a *different* (green/amber/slate) set — that set fails the validator as adjacent donut wedges but passes as small dots beside a text label.

- `AUDIT_ORDER` (`audit.status.ts:30`) is the single stacking sequence, re-exported by analytics as `AUDIT_STATE_ORDER` . Do not redeclare it — the colourblind check was run against that exact order, and the CASE in `auditRankSql` has no `ELSE` , so a new state missing from the array fails `audit.status.test.ts` rather than silently ranking `NULL` .

B21 — UI audit: accessibility, mobile and consistency

Problem. The UI has never had a systematic pass. Individual rules are well documented and well obeyed — the 44px tap floor, the validated chart palette, the `<dialog>` rule — but they were each written in response to a specific bug, and **nothing has checked the whole surface against them at once**. Three things make that gap larger than it looks:

1. **The test tooling cannot see any of it.** `npm run build` and the jsdom component tests have no layout engine, so tap targets, overflow, contrast and focus order are invisible to CI. The only instrument is a real browser, and nobody has driven one across the app on purpose.
2. **Two design systems coexist** (§3.7). The ledger pages and the Tailwind/shadcn pages were built years apart in effort terms, and no one has compared them for focus rings, error presentation, keyboard behaviour or heading order.
3. **Accessibility has never been assessed at all.** There is no axe/Lighthouse run recorded anywhere, no keyboard-only walkthrough, and no screen-reader pass. The one concrete signal in this handover is negative: the CSV import has **no `aria-live` busy announcement**, and its only `role="alert"` is the error paragraph (§B3). That is unlikely to be the only case. For a government system this is also a **Section 508 / WCAG 2.1 AA** exposure, not merely a quality issue.

Approach. Audit first, fix second — the output of this ticket is a prioritised defect list, not a redesign. Suggested order:

- **Automated sweep.** Run axe (or Lighthouse a11y) over each distinct page: `/` , `/login` , `/items` , `/i/<id>` , `/receipts/<n>` , `/receipts/new` , `/admin` , `/admin/analytics` , `/admin/queue` , `/admin/items/import` , `/account` . Record violations per page. This is cheap and catches contrast, labels, and landmark problems in bulk.
- **Mobile viewport pass.** At 375px, verify the 44px floor actually holds everywhere interactive — especially anywhere a height was overridden — and that no table or dialog forces horizontal scroll. `/items` and the receipt builder are the highest-risk pages because they are the densest.
- **Keyboard-only pass.** Tab through the receipt builder, the delete dialog, the suggestion comboboxes and the analytics filters. Focus must be visible, must not be trapped, and must return sensibly when a dialog closes.
- **Consistency pass across the two systems.** Compare a ledger page and a Tailwind page for focus ring, error message placement, disabled state, and heading hierarchy. Decide which is canonical and record it in §3.7 — this is a documentation output as much as a code one.

- **Then triage.** File the findings as their own small tickets rather than fixing everything inside this one. Anything that is a documented-rule violation (tap floor, palette, dialog) is a straightforward fix; anything that is a *missing* rule needs a decision first.

Files. Audit spans `src/app/**` and `src/components/**`; findings likely concentrate in `src/components/ui/*` (the shared primitives), `src/app/admin/items/import/ImportItemsForm.tsx` (the known `aria-live` gap) and the two densest pages, `src/app/items/` and `src/app/receipts/new/`. Any rule decided here belongs in **§3.7** and in `CLAUDE.md`.

Effort: M for the audit itself; the fixes are unknown until it runs, which is the point of splitting them out. **Priority:** P2 — no user is blocked today, but this is the kind of debt that gets more expensive per page added, and the Section 508 angle may make it non-optional on a timeline this handover cannot see.

Risks and dependencies. Low risk: the audit changes nothing. The real risk is scope creep — treat "the UI could look better" as **out of scope** and keep this to measurable rule violations, or it becomes an open-ended redesign nobody finishes. Depends on nothing; can start immediately. Pairs naturally with **B19** (broaden end-to-end browser coverage), since both need a driven browser and B19 could carry the regression tests this produces.

Data & performance

B6 — Configure the connection pool explicitly

Problem. `src/lib/prisma.ts` passes only a connection string:

```
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL });   // :7
export const prisma = globalForPrisma.prisma ?? new PrismaClient({ adapter });  // :9
if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;     // :11
```

Verified: `PrismaPg`'s constructor takes `pg.Pool | pg.PoolConfig | string` (`node_modules/@prisma/adapter-pg/dist/index.d.ts:42`), so it builds a `pg.Pool` from that object — and **no pool options are set**, so every value is a `pg` default. `pg`'s default `max` is **10** (`node_modules/pg/lib/defaults.js:42`). Nothing sets `idleTimeoutMillis` or `connectionTimeoutMillis` either.

Consequences worth understanding before changing anything:

- On Vercel, the module is evaluated once per server instance, so **each concurrent serverless instance can hold up to 10 connections** to the Supabase transaction pooler. The `globalThis` pin at `:11` is a **dev-only** HMR guard, not a production mechanism.
- `DATABASE_URL` must point at the **transaction pooler** (port 6543, `pgbouncer=true`) and `DIRECT_URL` at the **session/direct** connection (5432). `DEPLOY.md` records that mixing them up is the most common failure, and that a "prepared statement already exists" error is the signal to move `DATABASE_URL` to the session pooler — no code change needed, because the app uses the `pg` driver adapter rather than Prisma's own pooling.
- **Related:** finding **F4** shows one `/receipts/new` request can issue roughly 2,000 statements against that 10-connection pool — see B7. The pool is the resource that fan-out contends for.

Approach.

1. Set the pool explicitly:
   `new PrismaPg({ connectionString: process.env.DATABASE_URL, max: <N>, idleTimeoutMillis: …, connectionTimeoutMillis: … })`.
   For a serverless deployment behind a transaction pooler, a **small** `max` (2–5) is usually right — each instance needs concurrency only for the parallel queries a single request issues, and the pooler multiplexes across instances.
2. Make `N` configurable by environment variable so it can be tuned without a code change.
3. Measure before and after against Supabase, not locally. Latency is the whole question and local numbers do not transfer — `DEPLOY.md` §7a makes the same point about the import.
4. Document the chosen value and its reasoning in `DEPLOY.md`.

Files. `src/lib/prisma.ts:7`; `DEPLOY.md` §4 and the "Pooled vs direct" note; `.env.example`.

Effort: S. **Priority: P1** — it is a one-line change to an unexamined default in the hottest shared resource in the system.

Risks.

- ⚠️ **Too low a `max` serializes the app's deliberate parallelism.** The analytics dashboard and the cron worker both fire several queries with `Promise.all` on purpose; `/receipts/new` fires many. Setting `max: 1` would turn those into queues. Pick a number from measurement, not from a blog post.
- Sizing interacts with Supabase's own pooler client limit, which depends on the plan. Check the project's actual limit before raising `max`.
- This handover **could not observe production**, so the current real-world connection count is unknown. Step 3 is not optional.

B7 — Dedupe and batch the `/receipts/new` query fan-out (F4)

Problem. `src/app/receipts/new/page.tsx:13–16` splits `?items=` on commas with no cap and no dedupe, then issues **one `prisma.item.findUnique` per id, all concurrent** (`:16`). Line `:33` issues a **second** concurrent wave — `getLastReceiver` per surviving item, each of which is `getHoldingTransfer`: a `findFirst` with a nested `some` over `lines.items`, an `orderBy`, and a filtered `include`, **with no scalar `select`** — so it materializes the whole `Transfer` row *including the signature blob*.

Because there is no dedupe, repeating one known-good id N times produces N full holder lookups — roughly 600 real ids and ~2,000 statements from a single HTTP GET, against a 10-connection pool. The `MAX_RECEIPT_ROWS` / `MAX_ITEMS_PER_ROW` guards at `:20–21` run **after** both waves, so they bound what is *persisted*, never what is *queried*. And no rate limit applies, because the proxy's 300/min budget is anonymous-only by design (`src/proxy.ts:208`).

This is the `Promise.all(ids.map(id => prisma…))` pattern `CLAUDE.md` bans by name, in a security-relevant position.

Approach. Cap and dedupe **before** any query — `[...new Set(ids)].slice(0, MAX_RECEIPT_ROWS * MAX_ITEMS_PER_ROW)` — then replace both fan-outs with the batched helpers that already exist:

- `getItemsByIds` (`src/modules/items/items.service.ts:51`) — already used by the qr-sheet route for this exact input shape;
- `holdersForItems` (`src/modules/transfers/holders.query.ts:32`) — resolves custody for a whole page in one statement and selects only `receiverName` .

Files. `src/app/receipts/new/page.tsx:13-16, :20-21, :31-39` .

Effort: S. **Priority:** P2 — authenticated-only and fully attributable, so the realistic harm is a trusted technician degrading their own team's tool. But the fix is small and both helpers already exist.

Risk. ⚠️ `holdersForItems` uses the **looser** shared custody rule (any open receipt), while `getLastReceiver` uses the **strict** latest-receipt rule that fails closed — and this page's value **prefills the sender on a signed DA 2062**. Swapping them changes which name is printed in the ambiguous case. Either keep `getLastReceiver` 's semantics with a batched implementation, or make the change deliberately and write down the decision.

Technical debt / refactoring

B17 — U9: close the `check-security-docs` guardrail's blind spots

Problem. `scripts/check-security-docs.mjs` is the mechanism that keeps `docs/SECURITY.md` honest, and it is the structural reason the drift this handover reports went unnoticed. Its `WATCHED` list (`:35-142`) does **not** cover:

- `prisma/` — which is why B12's missing trigger escaped notice entirely;
- `src/modules/transfers/seal.ts` — the file that defines *what the Ed25519 signature binds*. Note `src/lib/crypto.ts` **is** watched (`:88`) while `seal.ts` is not, so the manifest can be narrowed without the guardrail firing;
- `.github/workflows/purge-cron.yml` (only `ci.yml` is watched, `:132`);
- **itself** — so the watch list can be edited in a passing PR.

Additionally, **its own guard test never runs in CI** — the three CI jobs are `sast` , `security-docs` and `build` , with no vitest step.

Approach. Add the four regexes to `WATCHED` ; run `scripts/check-security-docs.test.mjs` in CI (naturally solved by B18).

Files. `scripts/check-security-docs.mjs:35-142` ; `.github/workflows/ci.yml` .

Effort: S (about 30 minutes). **Priority: P1** — `SECURITY_ASSESSMENT.md:374` calls this "the highest-leverage process item in the report", and the assessment's concluding paragraph names fixing it as *the single change most likely to keep the report's conclusions true a year from now*.

Risk. Watching `prisma/` broadly will fire on every routine migration, which trains people to reach for the sanctioned `[skip security-doc]` bypass and blunts the guardrail. Consider watching the security-relevant slice narrowly — a `prisma/manual/` directory rule plus a pattern

matching `POLICY|ROW LEVEL SECURITY|GRANT|REVOKE|EVENT TRIGGER` in migration content — rather than the whole directory.

B22 — Resolve the stranded `@testing-library/jest-dom` dependency

Problem. `@testing-library/jest-dom@^7.0.0` is in `devDependencies` and installed in `node_modules`, but **nothing imports it and it is not wired up**. It is not in `vitest.config.ts`'s `setupFiles` (which loads only `tests/helpers/setup-env.ts`), so its matchers — `toBeInTheDocument()`, `toBeDisabled()`, `toHaveAttribute()` — do not exist even for a test that tries to use them.

Meanwhile **two test files carry comments asserting it is not installed** and work around its absence:

- `src/app/login/LoginForm.test.tsx:36` — *"jest-dom is not installed here, so assert on the DOM property directly."*
- `src/components/SuggestCombobox.test.tsx:7` — the same note.

So the repository now contains a dependency that is present but unusable, and comments that say it is absent. Both cannot be right, and whichever a future reader believes, they lose. Provenance is unclear: the change was uncommitted in the working tree as of 2026-08-05 and predates that session, which is why it is not in any commit.

Approach. Pick one and finish it — do not leave the third state.

- **Finish it:** add `@testing-library/jest-dom/vitest` to `setupFiles` in `vitest.config.ts:31`, then delete the two comments and simplify the assertions they guard. Note the suite is `environment: "node"` with jsdom opted in per-file via a `// @vitest-environment jsdom` docblock, so the setup import must tolerate running under the node environment too, or be scoped to the component tests.
- **Revert it:** `git checkout package.json package-lock.json && npm install`. The two comments then become true again and nothing else changes.

Files. `package.json`, `package-lock.json`, `vitest.config.ts:31`, `src/app/login/LoginForm.test.tsx:36`, `src/components/SuggestCombobox.test.tsx:7`.
Effort: S. **Priority:** P3 — no runtime impact whatsoever; this is purely about the next reader not being misled. Worth doing alongside **B18**, since running the suite in CI is the change most likely to expose whichever choice is made here.

B18 — Run the test suite in CI

Problem. `.github/workflows/ci.yml` defines exactly three jobs — `sast`, `security-docs`, `build` — and **none of them runs** `npm test`. The Vitest suite (116 test files) holds this codebase's most important invariants, including `readiness.parity.test.ts` and `items.readiness-sort.parity.test.ts`, whose whole purpose is to fail when someone changes one of a pair of twinned implementations. Today they fail **only if a human remembers to run them**.

Approach. Add a `test` job with a `postgres:16` service container, run `prisma migrate deploy` against it, then `npm test`. Add it to the required-checks list on `main` alongside the existing three.

Files. `.github/workflows/ci.yml`; branch-protection settings (required checks); `CLAUDE.md` and `docs/ARCHITECTURE.md:309–312`, both of which enumerate "three required checks" and would become wrong.

Effort: S. **Priority: P1**.

Risks.

- The suite is written against a **real migrated Postgres** and truncates between tests, so it needs a service container, not a mock. That also means it cannot be trivially parallelised across jobs without giving each its own database.
- Adding a fourth required check makes merges slower and can wedge an urgent fix. `enforce_admins` is already `false`, so an emergency bypass exists.
- Expect to fix a handful of environment-dependent tests on first run.

B19 — Broaden end-to-end browser coverage

Problem. `tests/e2e/` contains exactly **one** spec file, `auth.spec.ts`. Playwright is configured and works, and `npm run db:seed:e2e` seeds fixtures for it — but the coverage is authentication only.

This matters more here than in most codebases, for a documented reason: **neither `npm run build` nor jsdom has a layout engine**, so neither is evidence for a CSS or mobile change. The `<dialog>` bug in §3.11 — 50 invisible boxes eating clicks — shipped and was caught only in a real browser. A browser-driven suite is the *only* automated thing that could have caught it.

Approach. Add specs for the flows whose failure is most expensive:

1. Create a hand receipt end to end, including the signature pad, and assert the PDF route returns a PDF.
2. Process a partial return, then a full return, and assert the receipt closes and becomes immutable.
3. The `/items` list: search, UIC filter, compound sort, pagination — the surface with two query paths behind it.
4. The public path: PIN gate → unlock → item page → receipt page → PDF, and the receipt-link token bypass.
5. Mobile viewport smoke tests for the pages with a documented 44px touch-target floor.

Files. `tests/e2e/`; `playwright.config.ts`; `prisma/seed-e2e.ts` (fixtures).

Effort: M. **Priority:** P2.

Risk. ⚠️ Turnstile refuses automated browsers, including Playwright. `playwright.config.ts` pins Cloudflare's always-pass test keys for the server it starts; with real keys the e2e sign-in hangs at "Checking your browser…" and looks like a broken login. Never

respond to that by weakening the challenge. Also: the e2e suite shares the same test-database constraint — one runner at a time.

---

5. Known unknowns — what this handover could not verify

Stated explicitly, so nobody mistakes silence for confirmation.

1. **Nothing was executed.** No build, no test run, no dev server, no database connection, no HTTP request. Every claim above is from reading source, configuration, migrations and documentation in this repository.
2. **No production or deployed-environment access.** Specifically unverified:
   - whether `public.rls_auto_enable()` and its event trigger actually exist in the Supabase database (this is the whole point of B12);
   - whether `PublicAccessSetting` and `DeviceCategory` carry anon grants;
   - whether a legacy `admin@example.com` account survives from the 2026-06-30 → 07-06 window (see B20);
   - which environment variables are actually set in Vercel — whether Turnstile keys, the Upstash pair, or a complete `GMAIL_*` set are live is asserted by `docs/SECURITY.md` and `DEPLOY.md`, not observed;
   - the real connection count and pool behaviour under load (B6).
3. **Effort estimates are judgement, not measurement.** Treat S/M/L as relative ordering.
4. **Production data figures are quoted from the project's own documents**, not from a database: the 1,139-item and 1,053-device counts, the "31 audited rows with 31 distinct stamps" observation, and the analytics UIC-vs-unit cardinality figures all come from `CLAUDE.md` and `docs/ARCHITECTURE.md`.
5. **The CodeGraph index lags the working tree.** Several files present on disk (`src/app/api/items/import/route.ts`, `src/app/admin/items/qr-sheet/`, `src/app/privacy/`, `src/app/terms/`) were absent from the index, and several scratch/probe test files *in* the index no longer exist on disk. Every structural claim in this document was confirmed against the filesystem. If you use CodeGraph, re-index first and verify surprising hits.
6. `docs/SECURITY.md` **was read selectively** — its "At a glance" table, §9, §10, the section index, and the specific lines cited by `SECURITY_ASSESSMENT.md`. It is 1,615 lines and repays reading in full.
7. `CHANGELOG.md` **was read for the two most recent months.** It is 925 lines and the earlier entries carry rationale not repeated elsewhere.
8. **The security findings are restated from** `SECURITY_ASSESSMENT.md`, whose own limitations (§10 of that document) apply transitively here. Where this handover independently confirmed something — the absent RLS DDL, the absent `TransferItem` unique constraint, the `pg` pool default, the `PrismaPg` constructor signature, the absent CI test job, the guardrail's watch list — it says so.

Written 2026-08-05 against branch `feat/receipt-link-pin-bypass`. If you are reading this more than a few months later, treat the file:line citations as starting points rather than addresses, and re-derive anything load-bearing.